

Télétravailler sans s'isoler : Conception d'une plateforme numérique de mise en relation de télétravailleurs

PROJET D'INTÉGRATION

MARIN LOÏC

PSR12864

Table des matières

1. Introduction.....	5
2. Contexte du projet	5
2.1. Situation actuelle	5
2.2. Problématique	6
3. Analyse des besoins.....	7
3.1. User stories	8
3.1.1. Authentification	8
3.1.2. Hôte - Gestion de mon espace et de mes créneaux.....	9
3.1.3. Invité – Recherche et réservation de créneaux	9
3.1.4. Evaluation	9
3.1.5. Communication	10
3.2. Besoins non fonctionnels	10
3.3. Analyse concurrentielle	10
4. Contraintes métiers	12
4.1. Utilisateur.....	12
4.2. Espace	12
4.3. Créneaux.....	12
4.4. Réservation	13
4.5. Avis	13
4.6. Equipement	13
4.7. Règles	14
4.8. Notification.....	14
5. Use cases.....	15
5.1. Use case diagram.....	15
5.2. Use cases.....	16
5.2.1. Créer un compte	16
5.2.2. S’identifier	17
5.2.3. Récupérer son mot de passe.....	17
5.2.4. Se déconnecter.....	19
5.2.5. Modifier son compte.....	19
5.2.6. Créer son espace de travail.....	20

5.2.7.	Modifier son espace de travail	21
5.2.8.	Créer un créneau horaire	21
5.2.9.	Modifier son créneau horaire.....	22
5.2.10.	Annuler son créneau horaire	23
5.2.11.	Consulter les créneaux disponibles	24
5.2.12.	Faire une réservation	24
5.2.13.	Consulter ses réservations.....	25
5.2.14.	Modifier sa réservation	25
5.2.15.	Annuler sa réservation	26
5.2.16.	Evaluer sa réservation	27
5.2.17.	Modifier son évaluation.....	28
5.2.18.	Consulter ses évaluations.....	28
6.	Limites du projet	30
7.	Modélisation des données	31
7.1.	Schéma entité-relation	31
7.2.	Schéma relationnel	32
7.3.	Entités.....	33
7.3.1.	Users.....	33
7.3.2.	RefreshTokens.....	33
7.3.3.	Workspaces.....	33
7.3.4.	Slots.....	33
7.3.5.	Bookings.....	34
7.3.6.	Messages	34
7.3.7.	Reviews	34
7.3.8.	Rules.....	34
7.3.9.	WorkspaceRules	34
7.3.10.	Materials	35
7.3.11.	WorkspaceMaterials.....	35
7.3.12.	BookingMaterials	35
8.	Décisions technologiques	36
8.1.	Détermination des supports	36
8.2.	Choix de framework et langage.....	36

8.3.	Backend	39
8.3.1.	La clean Architecture.....	39
8.3.2.	CQRS	40
8.3.3.	Les Domain Events et le système de notification	41
8.3.4.	La gestion des erreurs métiers et le Result Pattern	42
8.3.5.	La validation des entrées et FluentValidation.....	43
8.3.6.	La base de données.....	43
8.3.7.	Les tests unitaires, Xunit et Mock.....	44
8.3.8.	Les websockets et SignalR	44
8.4.	Front End.....	45
8.4.1.	Langage et Architecture frontend.....	45
8.4.2.	La bibliothèque MudBlazor	46
8.5.	Sécurité.....	47
8.6.	Méthodologie de travail	48
8.7.	Déploiement.....	49
9.	Considérations budgétaires	50
10.	Vue et écrans	51
11.	Conclusion	53
12.	Bibliographie	54
13.	Annexes	56
	Annexe 1 : Maquettes.....	56
	Annexe 2 : Diagramme d'authentification	61
	Annexe 3 : Analyse MoSCoW	63
	Annexe 4 : Diagramme de classe.....	66
	Annexe 5 : Enquête sur le télétravail et la santé mentale	67

Remerciements :

Je tiens tout particulièrement à remercier Monsieur Dauvin qui m'a fourni de très précieux conseils lors de l'écriture de ce travail.

Je suis également très reconnaissant de la patience de ma compagne qui m'a soutenu du mieux qu'elle pouvait pendant ces 4 longues années d'études.

Je remercie de même ma maman qui a pris le temps de lire et relire ce travail et d'y relever tout ce qui pouvait être corrigé. Merci encore d'avoir gardé mon fils pendant les longues soirées de cours.

Je tenais aussi à remercier tous mes professeurs qui, en transmettant leur savoir et leur expérience, ont grandement renforcé mon esprit critique et enrichi mes connaissances théoriques. Je suis certain que ces connaissances me seront bien utiles dans le futur.

1. Introduction

Quand le travail hybride est devenu la norme pour un nombre important de métiers, comment permettre aux télétravailleurs de réduire leur isolement tout en profitant des avantages du télétravail ?

Tel est le défi auquel ce travail va tenter de répondre. Tout d'abord, nous commencerons par dresser un état des lieux du télétravail, de ses avantages et de ses effets sur l'isolement social. De cette analyse découlera la problématique et la manière dont le numérique peut y apporter une réponse.

Ensuite, nous définirons les besoins au travers de personas, de « user stories » et de contraintes, avant d'étudier le modèle de données et les technologies retenues. Une attention particulière sera portée à l'expérience utilisateur.

Enfin, nous dresserons un bilan pour évaluer si la solution proposée répond à la problématique et pour identifier les pistes d'améliorations futures.

2. Contexte du projet

2.1. Situation actuelle

C'est à l'étranger que, dans les années 1970, quelques entreprises pionnières ont commencé à mettre en place le télétravail. En Belgique, avant la crise du Covid, seulement 15 à 20% des travailleurs étaient concernés par le télétravail qui se faisait le plus souvent de manière occasionnelle. Lors de cette crise en mars 2020, le télétravail est devenu la norme dans le secteur privé pour la majorité des métiers pouvant se faire à distance avec un matériel limité tel qu'un ordinateur portable. Le secteur public est beaucoup moins concerné, probablement en raison d'une plus grande réticence face au changement.

Parmi les points positifs, nous pouvons citer notamment une réduction du temps de trajet et du stress en permettant aux employés de faire moins de route et de travailler dans un cadre plus propice à une bonne santé mentale. Un autre des bénéfices est la réduction des coûts en essence ou coûts potentiels liés à une location dans un espace de coworking. Le télétravail bénéficie aussi à l'employeur en termes de coûts d'entretien des locaux. De même, en ce qui concerne la réduction des bouchons sur les routes en évitant aux employés de devoir effectuer des déplacements parfois importants pour rejoindre des espaces de coworking ou les bureaux de la société et par conséquent une réduction de la pollution liée à ces déplacements.

2.2. Problématique

Le télétravail apporte du positif comme nous l'avons vu. Néanmoins, il apporte aussi une problématique très claire : l'isolement des travailleurs.

Le Service public fédéral Emploi, Travail, et Concertation sociale, bien que confirmant certains bienfaits associés au télétravail, met en avant un risque accru de troubles psychosociaux tels que de l'isolement ou de la démotivation ainsi qu'une limite parfois trop floue entre vie professionnelle et vie privée. Ce dernier insiste sur le fait qu'il est recommandé que l'employeur prenne des mesures pour éviter l'isolement des travailleurs et veille au maintien des relations sociales, en particulier pour les employés les plus vulnérables de l'entreprise. Une autre étude du Centre de connaissance belge sur le bien-être au travail confirme que la proportion de télétravailleurs souffrant d'un sentiment d'anxiété est deux fois plus élevée que celle des non-télétravailleurs.

Plus loin de chez nous, une étude menée aux Etats unis a révélé que la solitude des travailleurs pouvait avoir un impact considérable sur la santé physique et mentale quand le travailleur est isolé plus de 3 jours par semaine. Parmi ces effets négatifs, on peut noter notamment des résultats cardio-vasculaires moins bons que pour une personne travaillant en compagnie d'autres personnes mais aussi un risque plus important de développer une dépression ou de l'anxiété.

Dans cette même étude, le manque d'interaction sociale est mis en cause. Les contacts et discussions informelles, autour de la machine à café par exemple, apportent un soutien moral et aident les employés à gérer leur stress.

Face à cet isolement, il existe plusieurs alternatives. Le retour au bureau en est une mais elle implique la perte des bénéfices précités : temps de trajets, pollution, stress de la route, etc. Les espaces de coworking constituent une autre alternative mais sont coûteux et manquent de flexibilité. De plus, ils sont peu présents en dehors des grandes villes. Pour ce qui est de travailler dans un café, une bibliothèque ou tout autre endroit, on peut vite être limité en termes de confort, de calme ou d'infrastructure.

Lors d'une autre étude menée récemment auprès d'un public de 30 à 40 ans à Bruxelles et ses environs, plus de la moitié des personnes interrogées ressentaient parfois un isolement lors du télétravail et pourtant la plupart n'envisageaient pas un espace de coworking pour autant. Les raisons invoquées étaient variées mais le coût, la distance et l'accessibilité étaient celles qui revenaient le plus fréquemment.

Sur base de ces constatations, il est intéressant de se poser la question suivante : comment mettre en place une solution numérique de proximité permettant aux télétravailleurs de bénéficier des bienfaits du télétravail tout en apportant une solution à l'isolement social qu'ils peuvent subir et ses conséquences ?

3. Analyse des besoins

Pour répondre à la question soulevée dans la problématique, nous devons d'abord préciser ce que devrait apporter une telle plateforme aux personnes concernées. Pour réaliser cette analyse, nous allons recourir à la mise en place de 2 personas : Thomas et Sophie.

Sophie est une jeune travailleuse dans le domaine de l'informatique. Elle habite près de Verviers. Elle a 30 ans et possède une grande maison. Elle est en télétravail du lundi au jeudi et se rend au bureau les vendredis. Son mari, Jean, travaille à domicile le mercredi et également dans le domaine de l'informatique.

Sophie partage un grand bureau avec Jean. Quand il n'est pas là, les lundi, mardi et jeudi, elle ressent un vide et souffre d'anxiété. Elle souhaiterait partager cet espace avec un autre travailleur les jours d'absence de son mari.

Un mardi matin, elle décide de chercher une solution depuis son ordinateur et découvre WorkTogetherly. Elle doit créer un compte avec son adresse électronique. En quelques clics, son compte est créé et elle peut configurer son espace en mettant une photo de l'espace de travail, une description et son adresse. Elle préfère préciser que c'est un espace non-fumeur mais indique qu'un écran sera disponible si besoin et que les animaux de compagnie sont acceptés.

Dans la foulée, elle crée un créneau horaire pour le jeudi. Dans l'heure, une personne, Fabrice, réserve l'espace de travail et Sophie en est immédiatement notifiée. Peu de temps après, Sophie sera à nouveau notifiée lorsque Fabrice, ayant un contretemps, annulera sa réservation. Heureusement, plus tard dans la journée, un autre utilisateur du système, Thomas, prendra sa place.

Sophie peut à tout moment voir les prochaines réservations et se préparer à recevoir du monde. Elle garde le contrôle et peut annuler ou modifier ses créneaux à tout moment, guidée par le site qui lui évite de faire de la surréservation.

Avec le temps, les avis lui ont fait comprendre que la tasse de café et le croissant proposés aux invités étaient vraiment bien vus !

Thomas est un comptable de 52 ans et travaille 3 jours par semaine dans son petit appartement : les mardi, mercredi et jeudi. Le reste du temps, il doit se rendre à son bureau. Il n'aime pas spécialement le télétravail mais habite lui aussi près de Verviers alors que son bureau est à Bruxelles : ça fait une longue route !

Malheureusement son budget est limité et il ne peut pas se permettre d'aller tous les jours à l'espace de coworking proche de chez lui. Mercredi, en promenant son chien tout

en navigant sur son téléphone, il découvre une application mobile permettant de trouver d'autres télétravailleurs dans sa situation et il n'hésite pas à l'installer.

Il crée son compte depuis son téléphone mais, comme à son habitude, oublie de noter son mot de passe. Une fois de retour chez lui, il récupère ce dernier grâce à l'application. Il effectue facilement une recherche autour de sa position et voit que Sophie propose justement un espace de travail près de chez lui le lendemain. Il se dépêche de réserver en précisant qu'il pense arriver pour 8h30 et remarque avec satisfaction que son chien est accepté également et qu'il peut même avoir un écran à disposition.

Jeudi, il est accueilli par Sophie avec café et croissant à 8h30. Ils travaillent toute la journée et ils peuvent discuter quand ils font des pauses. Thomas termine sa journée et s'empresse de laisser un avis positif en mentionnant la qualité de l'accueil.

Grâce à ces personas, nous pouvons commencer à nous faire une idée des fonctionnalités qui devraient être présentes pour répondre aux besoins des télétravailleurs isolés.

3.1. User stories

Maintenant que nous avons identifié certains besoins grâce aux parcours de Thomas et de Sophie, nous allons exprimer ces besoins de manière plus formelle en les reformulant sous forme de « User Stories » regroupées dans 5 grands thèmes : Authentification, hôte, invité, évaluation et communication.

3.1.1. Authentification

US01 : En tant que visiteur, je veux créer un compte avec mon adresse électronique et mon mot de passe afin d'accéder à la plateforme.

US02 : En tant qu'utilisateur, je veux me connecter avec mon adresse électronique et mon mot de passe afin d'accéder à mon espace personnel et aux fonctionnalités de la plateforme.

US03 : En tant qu'utilisateur, je veux réinitialiser mon mot de passe afin de ne pas perdre l'accès à mon compte.

US04 : En tant qu'utilisateur, je veux pouvoir modifier mes informations personnelles afin de mettre à jour mon profil.

3.1.2. Hôte - Gestion de mon espace et de mes créneaux

US05 : En tant qu'hôte, je veux créer un espace de travail avec une description, des caractéristiques, une localisation (manuelle ou géolocalisée), une liste de règles et d'équipements, afin que les invités potentiels sachent ce que je propose.

US06 : En tant qu'hôte, je veux définir des créneaux de disponibilités pour mon espace, afin de définir quand ils peuvent être réservés.

US07 : En tant qu'hôte, je veux définir le nombre de places disponibles afin d'éviter les surréservations.

US08 : En tant qu'hôte, je veux définir le type d'invités que j'accepte (fumeurs, animaux de compagnie), afin de m'assurer que les invités correspondent à mes critères.

US09 : En tant qu'hôte, je veux consulter la liste de mes réservations à venir, afin de me préparer à accueillir mes invités.

3.1.3. Invité – Recherche et réservation de créneaux

US10 : En tant qu'invité, je veux rechercher un espace de travail autour d'une position (géolocalisée ou entrée manuellement) afin de trouver un espace correspondant à mes critères.

US11 : En tant qu'invité, je veux voir les résultats sur une liste et/ou sur une carte, afin de déterminer les espaces les plus proches de moi.

US12 : En tant qu'invité, je veux consulter les détails d'un espace afin de déterminer si cet espace me convient.

US13 : En tant qu'invité, je veux réserver un créneau afin de confirmer ma venue.

US14 : En tant qu'invité, je veux annuler une réservation future afin de libérer la place.

US15 : En tant qu'invité, je veux consulter mes réservations à venir et passées, afin de suivre mon historique.

3.1.4. Evaluation

US16 : En tant qu'invité, je veux laisser un avis sur mes précédentes réservations afin d'informer les autres utilisateurs.

US17 : En tant qu'hôte, je veux consulter les avis sur mon espace afin de savoir comment je suis évalué.

3.1.5. Communication

US18 : En tant qu'invité, je veux être notifié de tout changement sur ma réservation pour savoir si une place s'est libérée.

US19 : En tant qu'hôte, je veux être notifié de tout changement sur ma réservation pour savoir si je dois trouver un autre espace de travail.

3.2. Besoins non fonctionnels

En plus de répondre à ces besoins par module, il est important que la solution réponde à certaines exigences transversales. Tout d'abord, comme nous l'avons vu avec Sophie et Thomas, l'application doit pouvoir être utilisée depuis un téléphone au travers d'une application mobile mais aussi depuis un site web. Cela a deux implications : la solution doit d'une part être « responsive » et proposer une interface adaptée aux ordinateurs de bureau comme aux téléphones et à la souris comme aux écrans tactiles. D'autre part, la base de données devra être partagée entre toutes les instances.

Pour que la meilleure expérience soit offerte à Sophie et Thomas, il faudra faire les opérations de manière asynchrone tant que cela sera possible et proposer une connexion sécurisée et persistante pour éviter de devoir se reconnecter à chaque utilisation du service.

3.3. Analyse concurrentielle

Maintenant que nous avons une idée claire de ce que devrait offrir une telle solution, il est essentiel d'analyser ce que propose actuellement la concurrence et qui, de plus, réponde à cette problématique.

Aucun produit couvrant exactement les mêmes besoins, c'est-à-dire une solution permettant de mettre en relation des télétravailleurs dans le but précis de leur permettre de télétravailler ensemble, n'a été mis en évidence lors de cette recherche.

Il existe un certain nombre de concurrents indirects à ce jour qui ciblent un marché plus ou moins différent. Parmi ceux-ci, nous pouvons penser aux applications de mises en relation de particuliers telles que NextDoor, Meetup, Bumble BFF. Ces applications ne se présentent pas vraiment comme une concurrence directe car elles sont centrées sur la mise en relation de particuliers mais pas dans le but de travailler ensemble. Une autre alternative serait d'utiliser les réseaux sociaux ou les forums : des annonces sur des réseaux sociaux cherchant d'autres télétravailleurs pour se rassembler lors du télétravail par exemple. Nous trouvons aussi des plateformes de coworking telles que WeWork,

Servcorp, Regus,... qui présentent une concurrence sérieuse si et seulement si l'utilisateur potentiel habite à proximité de l'un d'eux. Comme vu précédemment, il s'agit souvent de solutions coûteuses. Enfin, nous trouvons aussi certaines applications qui mettent en relation des indépendants. Parmi celles-ci, nous avons BuddyWorkers qui présente une solution proche dans le fonctionnement mais qui s'adresse avant tout à des indépendants. A ce jour, BuddyWorker ne propose pas d'application mobile.

En termes de solutions de substitutions, travailler dans un café, une bibliothèque ou tout autre lieu public ou tout simplement se rendre au bureau sont des solutions viables.

4. Contraintes métiers

Dans ce chapitre, nous passerons en revue les différentes règles mises. Certaines règles découlent directement des user stories tandis que d'autres sont ajoutées pour ajouter une cohérence à la solution.

4.1. Utilisateur

RM-US-1 : La combinaison email/mot de passe doit être correcte pour se connecter.

RM-US-2 : L'adresse électronique est unique.

RM-US-3 : Seul un utilisateur authentifié peut créer un espace, un créneau ou une réservation.

RM-US-4 : Un utilisateur peut être hôte et invité à tour de rôle avec un même profil.

4.2. Espace

RM-WS-1 : Un utilisateur ne peut avoir qu'un seul espace.

RM-WS-2 : Seul le propriétaire d'un espace peut le modifier.

RM-WS-3 : L'espace doit avoir une adresse valide permettant de déterminer une latitude et une longitude valides.

4.3. Créneaux

RM-SL-1 : Un créneau doit toujours être lié à un et un seul espace.

RM-SL-2 : Un utilisateur peut créer autant de créneaux qu'il le souhaite sur un espace.

RM-SL-3 : Un créneau a une heure de début et de fin et une date.

RM-SL-4 : La capacité d'un créneau doit être un nombre strictement positif.

RM-SL-5 : Un créneau contient par défaut autant de places que l'espace mais peut en contenir un nombre différent.

RM-SL-6 : Un créneau ne peut être créé que sur l'espace du propriétaire.

RM-SL-7 : Un créneau ne peut pas être réservé plus de fois qu'il n'y a de place dans celui-ci.

RM-SL-8 : Un créneau ne peut être modifié ou annulé que par son auteur.

RM-SL-9 : Un créneau dont l'heure de début est passée ne peut plus être modifié.

RM-SL-10 : Deux créneaux ne peuvent pas se superposer sur un espace de de travail

4.4. Réservation

RM-BO-1 : Une réservation doit correspondre à un créneau et un espace de travail.

RM-BO-2 : Une réservation ne peut être faite que pour un créneau dont la fin n'est pas encore passée.

RM-BO-3 : Il est possible de réserver un créneau qui a déjà commencé.

RM-BO-4 : Une réservation ne peut être modifiée que par son auteur.

RM-BO-5 : Une réservation ne peut plus être annulée une fois que son heure d'arrivée est passée.

4.5. Avis

RM-RV-1 : Un avis ne peut être laissé que sur un créneau dont la date est dépassée.

RM-RV-2 : La note doit être comprise entre 1 et 5.

RM-RV-3 : Un utilisateur ne peut laisser qu'un seul avis par Workspace, toutes réservations confondues.

RM-RV-4 : Un avis ne peut être laissé que par l'invité qui a fait la réservation.

RM-RV-5 : Seul l'auteur d'un avis peut modifier celui-ci.

RM-RV-6 : Lors de l'ajout d'un commentaire, la note est requise, pas le commentaire.

4.6. Equipement

RM-EQ-1 : Un équipement peut être lié à un ou plusieurs espaces de travail.

RM-EQ-2 : Un équipement peut être disponible dans un espace de travail dans des quantités différentes et spécifiables.

RM-EQ-3 : Un équipement ne peut être utilisé plus de fois dans un créneau que ne le spécifie sa quantité dans l'espace de travail.

RM-EQ-4 : Les équipements sont définis par le système (liste fermée) et l'hôte ne peut pas en créer librement.

4.7. Règles

RM-RU-1 : Une règle peut être liée à un ou plusieurs espaces de travail.

RM-RU-2 : Les règles sont définies par le système (liste fermée) et l'hôte ne peut pas en créer librement.

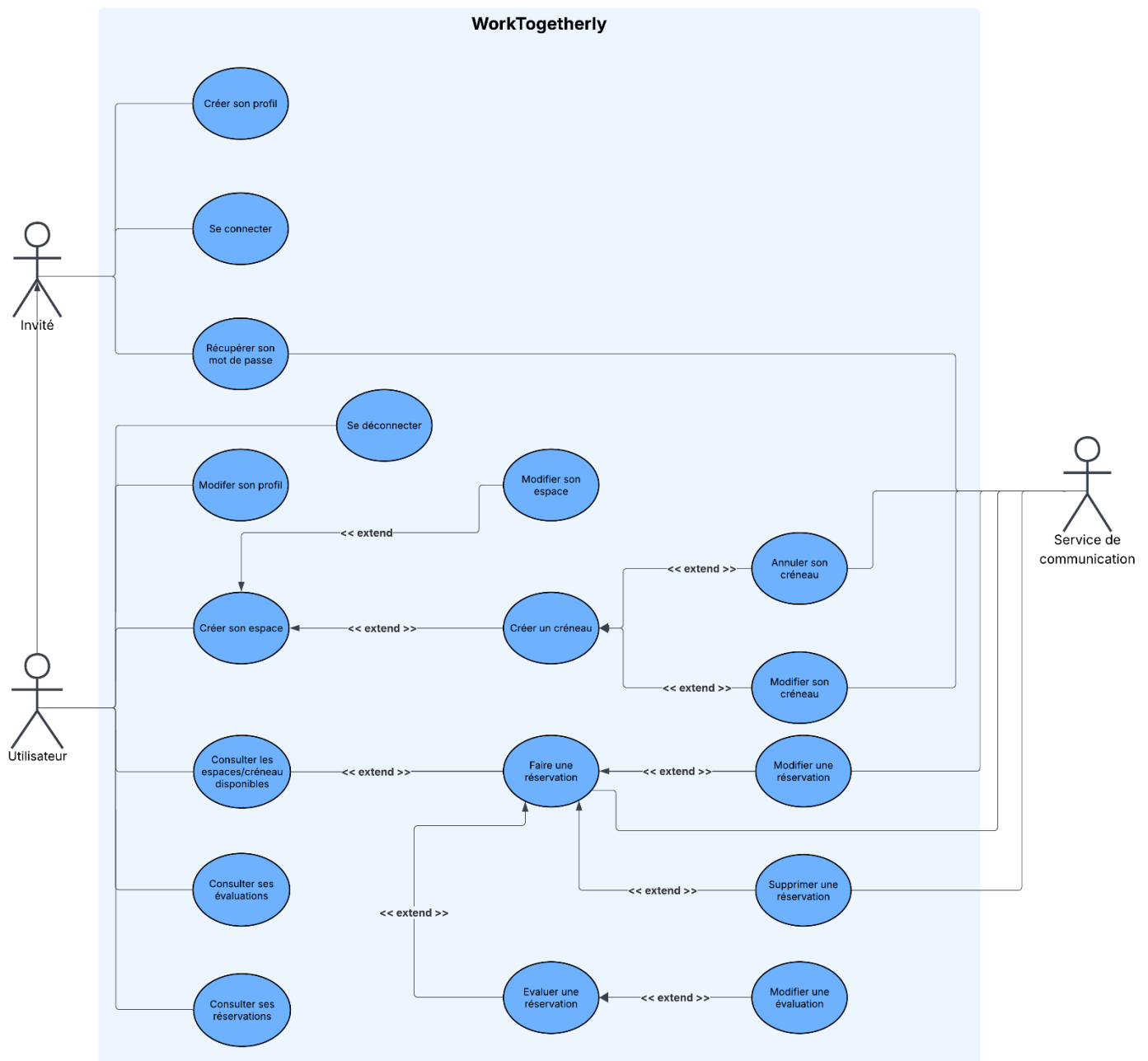
4.8. Notification

RM-NO-1 : Seul le destinataire d'une notification peut marquer cette dernière comme lue.

5. Use cases

5.1. Use case diagram

Après avoir clarifié les fonctionnalités nécessaires et des différentes contraintes mises en place, nous pouvons nous concentrer sur le lien entre les différents cas d'utilisation qui composeront la solution.



5.2. Use cases

Après avoir élaboré le «use case diagram», nous allons détailler chacun des cas d'utilisation l'un après l'autre en détail.

5.2.1. Créer un compte

Besoin fonctionnel : Ce cas d'utilisation doit permettre à une personne inconnue du système de se créer un compte et ainsi de devenir un utilisateur de la plateforme.

Acteur concerné : Une personne externe au système

Précondition :

- La personne ne possède pas de compte.
- La personne possède une adresse électronique valide.
- L'application est accessible.

Postconditions :

- La personne possède un compte et est donc connue du système. Elle est devenue utilisatrice du système.

Déroulement :

Scénario nominal :

1. L'invité se rend sur la page de création de compte.	2. Le système affiche la page de création de compte.
3. L'invité encode le mot de passe, l'adresse électronique qu'il utilisera pour se connecter et ses informations de base et soumet le formulaire.	4. Le système reçoit les informations et vérifie que l'adresse électronique est valide et n'est pas déjà utilisée. 5. Le système crée l'utilisateur dans le système.

Scénario alternatif :

La personne rentre une adresse électronique invalide ou déjà utilisée.

Le scénario alternatif démarre au point 4 du scénario nominal.

	1. Le système détecte des données invalides
	2. Le système indique à la personne l'erreur qui a empêché la création du compte.
	Le scénario nominal reprend au point 3

5.2.2. S'identifier

Besoin fonctionnel : Ce cas d'utilisation doit permettre à une personne connue du système de s'identifier.

Acteur concerné : Un utilisateur

Précondition :

- L'utilisateur connaît l'adresse électronique et le mot de passe à utiliser pour se connecter
- L'application est accessible.

Postconditions :

- L'utilisateur est connecté et reconnu par le système.

Déroulement :

Scénario nominal :

1. L'utilisateur non connecté se rend sur la page de connexion.	2. Le système affiche la page permettant de se connecter. Un champ courrier électronique et mot de passe est disponible pour cela.
3. L'utilisateur entre son login et son mot de passe.	4. Le système valide les informations.
	5. Le système redirige l'utilisateur vers l'espace connecté.

Scénario alternatif :

L'utilisateur non connecté entre une combinaison email/mot de passe invalide.

Le scénario alternatif démarre au point 3 du scénario nominal.

	1. Le système détecte des données invalides.
	2. Le système indique à l'utilisateur essayant de se connecter que l'adresse électronique ou le mot de passe sont incorrects.
	Le scénario nominal reprend au point 3

5.2.3. Récupérer son mot de passe

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de récupérer son mot de passe.

Acteur concerné : Un utilisateur, le service de communication.

Précondition :

- L'application est accessible.

Postconditions :

L'utilisateur a pu recréer un nouveau mot de passe pour accéder à son compte.

Déroulement :

Scénario nominal :

1. L'utilisateur non connecté se rend sur la page de récupération de mot de passe.	2. Le système affiche la page permettant de récupérer son mot de passe. Un champ adresse électronique est disponible pour cela.
3. L'utilisateur encode l'adresse électronique qu'il utilise pour se connecter au le système et soumet le formulaire.	4. Le système vérifie qu'il s'agit bien d'une adresse électronique valide. 5. Le système indique à l'utilisateur que si son adresse électronique est connue du système, un email lui sera envoyé pour réinitialiser son mot de passe. 6. Le système envoie un message électronique contenant un lien permettant de réinitialiser le mot de passe.
7. L'utilisateur clique sur le lien.	8. Le système redirige l'utilisateur vers une page permettant à l'utilisateur de choisir un nouveau mot de passe.
9. L'utilisateur choisit un nouveau mot de passe et le soumet.	10. Le système enregistre le mot de passe et redirige l'utilisateur vers la page de connexion où il pourra utiliser ses nouveaux identifiants.

Scénario d'erreur :

L'adresse électronique est inconnue du système.

Le scénario alternatif démarre au point 5 du scénario nominal.

	1. Le système détecte que l'adresse électronique n'est pas connue du système.
--	---

5.2.4. Se déconnecter

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de se déconnecter de l'application.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible
- L'utilisateur a déjà un compte dans le système
- L'utilisateur est connecté

Postconditions :

L'utilisateur a pu se déconnecter pour redevenir un invité.

Déroulement :

Scénario nominal :

1. L'utilisateur se rend dans la section pour se déconnecter.	2. Le système affiche l'option pour se déconnecter.
3. L'utilisateur lance le processus de déconnection.	4. Le système confirme l'action. 5. Le système prend les mesures nécessaires pour changer le statut de l'utilisateur connecté.

5.2.5. Modifier son compte

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de modifier les informations de son compte

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.
- L'utilisateur a déjà un compte dans le système.
- L'utilisateur est connecté.

Postconditions :

L'utilisateur a pu modifier les informations de son compte.

Déroulement :

Scénario nominal :

6. L'utilisateur se rend dans la section pour afficher les informations de son compte.	7. Le système charge et affiche le détail du compte de l'utilisateur.
8. L'utilisateur modifier les informations qu'il souhaite sur le formulaire contenant ses informations. 9. L'utilisateur soumet le formulaire.	10. Le système valide les informations. 11. Le système sauvegarde les informations. 12. Le système confirme la sauvegarde des informations à l'utilisateurs.

5.2.6. Créer son espace de travail

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de configurer son espace de travail.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur n'a pas encore créé son espace.

Postconditions :

L'utilisateur a pu créer son espace de travail.

Déroulement :

1. L'utilisateur se rend dans la section permettant de créer son espace de travail.	2. Le système affiche la page permettant de configurer son espace de travail.
	3. Le système invite l'utilisateur à créer son espace de travail. 4. Le système affiche un formulaire permettant de créer et configurer son espace de travail.
5. L'utilisateur remplit les informations nécessaires. 6. L'utilisateur soumet le formulaire.	7. Le système crée l'espace de travail de l'utilisateur en se basant sur les informations envoyées par l'utilisateur. 8. Le système confirme la création de l'espace de travail.

5.2.7. Modifier son espace de travail

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de configurer son espace de travail.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a déjà créé son espace de travail.
- L'utilisateur a déjà créé des créneaux.

Postconditions :

L'utilisateur a pu modifier son espace de travail.

Déroulement :

1. L'utilisateur se rend dans la section permettant de gérer son espace de travail.	2. Le système affiche la page permettant de configurer son espace de travail.
	3. Le système affiche un formulaire permettant de modifier son espace de travail.
4. L'utilisateur modifie les informations qu'il souhaite. 5. L'utilisateur soumet le formulaire.	6. Le système modifie l'espace de travail de l'utilisateur en se basant sur les informations envoyées par l'utilisateur. 7. Le système confirme la modification de l'espace de travail.

5.2.8. Créer un créneau horaire

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de créer un créneau horaire.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a déjà créé son espace de travail.

Postconditions :

L'utilisateur a pu créer un créneau horaire.

Déroulement :

1. L'utilisateur se rend dans la section permettant de créer un créneau horaire dans son espace de travail.	2. Le système affiche la page permettant de créer un créneau horaire dans l'espace de travail de l'utilisateur.
	3. Le système affiche un formulaire permettant de créer un créneau horaire.
4. L'utilisateur encode les informations permettant de créer le créneau horaire (heures de début, fin, date, nombre de places). 5. L'utilisateur soumet le formulaire.	6. Le système crée le créneau horaire de l'utilisateur. 7. Le système confirme la création du créneau et le rend disponible aux autres utilisateurs.

Scénario d'erreur :

Les dates ou l'heure du début sont dans le passé.

Le scénario alternatif démarre au point 5 du scénario nominal.

	6. Le système détecte que la date ou l'heure ne sont pas valides et demande à l'utilisateur de les corriger.
--	--

5.2.9. Modifier son créneau horaire

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de modifier son créneau horaire.

Acteur concerné : Un utilisateur, le service de communication

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a déjà créé son espace de travail.
- L'utilisateur a déjà créé son créneau horaire.

Postconditions :

L'utilisateur a pu modifier son créneau horaire.

Déroulement :

1. L'utilisateur se rend dans la section permettant de modifier son créneau horaire dans son espace de travail.	2. Le système affiche la page permettant de modifier un créneau horaire de l'utilisateur dans son espace de travail.
	3. Le système affiche un formulaire permettant de modifier le créneau horaire de l'utilisateur.
4. L'utilisateur modifie les informations de son créneau horaire dans le formulaire. 5. L'utilisateur soumet le formulaire.	6. Le système enregistre les modifications du créneau horaire de l'utilisateur. 7. Le système confirme la modification du créneau horaire. 8. Le système notifie les inscrits du changement.

5.2.10. Annuler son créneau horaire

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur d'annuler son créneau horaire.

Acteur concerné : Un utilisateur, le service de communication.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a déjà créé son espace de travail.
- L'utilisateur a déjà créé son créneau horaire.

Postconditions :

- L'utilisateur a pu annuler son créneau horaire.
- Les inscrits sont notifiés.

Déroulement :

1. L'utilisateur se rend dans la section contenant la liste de ses créneaux futurs dans son espace de travail.	2. Le système affiche la liste des créneaux de l'utilisateur.
3. L'utilisateur sélectionne l'option pour annuler le créneau.	4. Le système confirme la suppression du créneau. 5. Le système notifie les inscrits au créneau.

5.2.11. Consulter les créneaux disponibles

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de consulter les créneaux disponibles sur base de filtres.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.

Postconditions :

- L'utilisateur peut visualiser le détail des créneaux disponibles répondant à certains filtres.

Déroulement :

1. L'utilisateur se rend dans la section permettant de consulter les créneaux disponibles.	2. Le système affiche une série de filtres pour choisir le lieu et la date demandés.
3. L'utilisateur sélectionne le jour et le lieu qui l'intéresse.	4. Le système affiche les espaces disponibles sur une carte et dans une liste. 5. Le système charge les détails des créneaux disponibles pour les afficher à la demande de l'utilisateur.

5.2.12. Faire une réservation

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de réserver un créneau horaire dans un espace de travail.

Acteur concerné : Un utilisateur, le service de communication.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a effectué une recherche et a trouvé un créneau disponible.

Postconditions :

- L'utilisateur a réservé un créneau horaire en indiquant son heure d'arrivée prévue.

Déroulement :

1. L'utilisateur sélectionne l'espace qui lui convient.	2. Le système affiche les détails de l'espace de travail et les prochains créneaux de l'espace. 3. Le système affiche un bouton pour réserver en face de chaque espace avec des places restantes.
4. L'utilisateur réserve le créneau qui l'intéresse.	5. Le système valide que les conditions pour réserver le créneau sont remplies. 6. Le système enregistre la réservation. 7. Le système informe l'utilisateur du succès de la réservation. 8. Le système notifie l'hôte via le service de communication.

5.2.13. Consulter ses réservations

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de consulter ses réservations.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.

Postconditions :

- L'utilisateur peut gérer ses réservations futures ou passées.

Déroulement :

1. L'utilisateur se rend dans la section lui permettant de consulter ses réservations.	2. Le système affiche les réservations à venir des 7 prochains jours. 3. Le système affiche des filtres permettant à l'utilisateur de consulter les séances du passé ou du futur.
--	--

5.2.14. Modifier sa réservation

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de modifier sa réservation.

Acteur concerné : Un utilisateur

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a une réservation dans le futur.

Postconditions :

- L'utilisateur a modifié son créneau horaire.

Déroulement :

1. L'utilisateur se rend dans la liste de ses réservations.	2. Le système affiche la liste des prochaines réservations. 3. Le système affiche des boutons permettant de modifier ou annuler les réservations du futur.
4. L'utilisateur sélectionne la modification de la séance du futur.	5. Le système affiche un formulaire permettant de modifier la réservation.
6. L'utilisateur modifie les données dans le formulaire. 7. L'utilisateur soumet le formulaire.	8. Le système valide les informations. 9. Le système confirme la mise à jour de la réservation. 10. Le système notifie l'hôte via le service de communication.

5.2.15. Annuler sa réservation

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur d'annuler sa réservation.

Acteur concerné : Un utilisateur, le service de communication.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a une réservation dans le futur.

Postconditions :

- L'utilisateur a modifié son créneau horaire.

Déroulement :

1. L'utilisateur se rend dans la liste de ses réservations.	2. Le système affiche la liste des prochaines réservations.
---	---

	3. Le système affiche des boutons permettant de modifier ou annuler les réservations du futur.
4. L'utilisateur sélectionne la modification de la séance du futur.	5. Le système affiche un formulaire permettant de modifier la réservation.
6. L'utilisateur modifie les données dans le formulaire. 7. L'utilisateur soumet le formulaire.	8. Le système valide les informations. 9. Le système confirme la mise à jour de la réservation. 10. Le système notifie l'hôte.

5.2.16. Evaluer sa réservation

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur d'évaluer sa réservation.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a une réservation dans le passé.

Postconditions :

- L'utilisateur a évalué sa réservation.

Déroulement :

1. L'utilisateur se rend dans la liste de ses réservations.	2. Le système affiche la liste des prochaines réservations. 3. Le système affiche des boutons permettant de modifier ou annuler les réservations du futur.
4. L'utilisateur filtre les réservations pour afficher la réservation passée.	5. Le système affiche la réservation passée.
6. L'utilisateur sélectionne la réservation passée.	7. Le système affiche les détails de la réservation passée ainsi qu'une option pour laisser un avis.
8. L'utilisateur sélectionne l'option pour laisser un avis.	9. Le système affiche le formulaire pour laisser un avis.
10. L'utilisateur laisse une note/un commentaire. 11. L'utilisateur soumet le formulaire.	12. Le système vérifie les données du formulaire. 13. Le système enregistre l'évaluation.

	14. Le système informe l'utilisateur du succès de l'opération.
--	--

5.2.17. Modifier son évaluation

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de modifier une évaluation qu'il aurait faite par le passé.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.
- L'utilisateur est connecté.
- L'utilisateur a précédemment évalué un espace de travail.

Postconditions :

- L'utilisateur a modifié son évaluation.

Déroulement :

1. L'utilisateur se rend dans la liste de ses réservations.	2. Le système affiche la liste des prochaines réservations. 3. Le système affiche des boutons permettant de modifier ou annuler les réservations du futur.
4. L'utilisateur filtre les réservations pour afficher la réservation passée.	5. Le système affiche la réservation passée.
6. L'utilisateur sélectionne la réservation passée.	7. Le système affiche les détails de la réservation passée ainsi qu'une option pour modifier son avis.
8. L'utilisateur sélectionne l'option pour modifier son avis.	9. Le système affiche le formulaire pour modifier son avis.
10. L'utilisateur modifie sa note/son commentaire. 11. L'utilisateur soumet le formulaire.	12. Le système vérifie les données du formulaire. 13. Le système enregistre les modifications de l'évaluation. 14. Le système informe l'utilisateur du succès de l'opération.

5.2.18. Consulter ses évaluations

Besoin fonctionnel : Ce cas d'utilisation doit permettre à un utilisateur de consulter les évaluations laissées sur son espace de travail.

Acteur concerné : Un utilisateur.

Précondition :

- L'application est accessible.

Postconditions :

- N/A

Déroulement :

1. L'utilisateur se rend dans son espace de travail.	2. Le système affiche les détails de son espace de travail et ses évaluations.
--	--

6. Limites du projet

Etant donné qu'il s'agit ici d'un projet effectué dans le cadre d'un travail de fin d'étude, il est à noter que certaines considérations ont volontairement été écartées et devraient être prises en compte avant une éventuelle mise en production.

Parmi, celles-ci et pas des moindres sont les considérations GDPR ou encore la mise en place de conditions générales. Il va de soi, qu'il faudrait sans aucun doute insister auprès des utilisateurs sur le fait que certaines données personnelles seront visibles en ligne et pourraient être utilisées par des personnes mal intentionnées.

D'autre part, il a été notamment décidé de ne pas prendre en compte l'internationalisation de l'app. Il aurait pu être intéressant de la gérer ne fut ce que partiellement en procédant à l'utilisation de ressources traductibles dans le futur.

Enfin, l'aspect commercialisation ne sera pas approfondi dans cette version. Nous pouvons néanmoins imaginer plusieurs pistes telles que la mise en place d'un abonnement donnant accès à la plateforme ou à des fonctionnalités premium ou encore un système de paiement à la réservation avec une commission prélevée par la plateforme pour rentabiliser le coût de l'infrastructure.

7. Modélisation des données

7.1. Schéma entité-relation

Grâce à l'analyse, nous pouvons mettre en place un plan des différentes entités déterminantes. Pour des raisons de clarté, ce schéma est simplifié pour mettre en avant uniquement les relations et les entités et pas les champs qui les composeront.

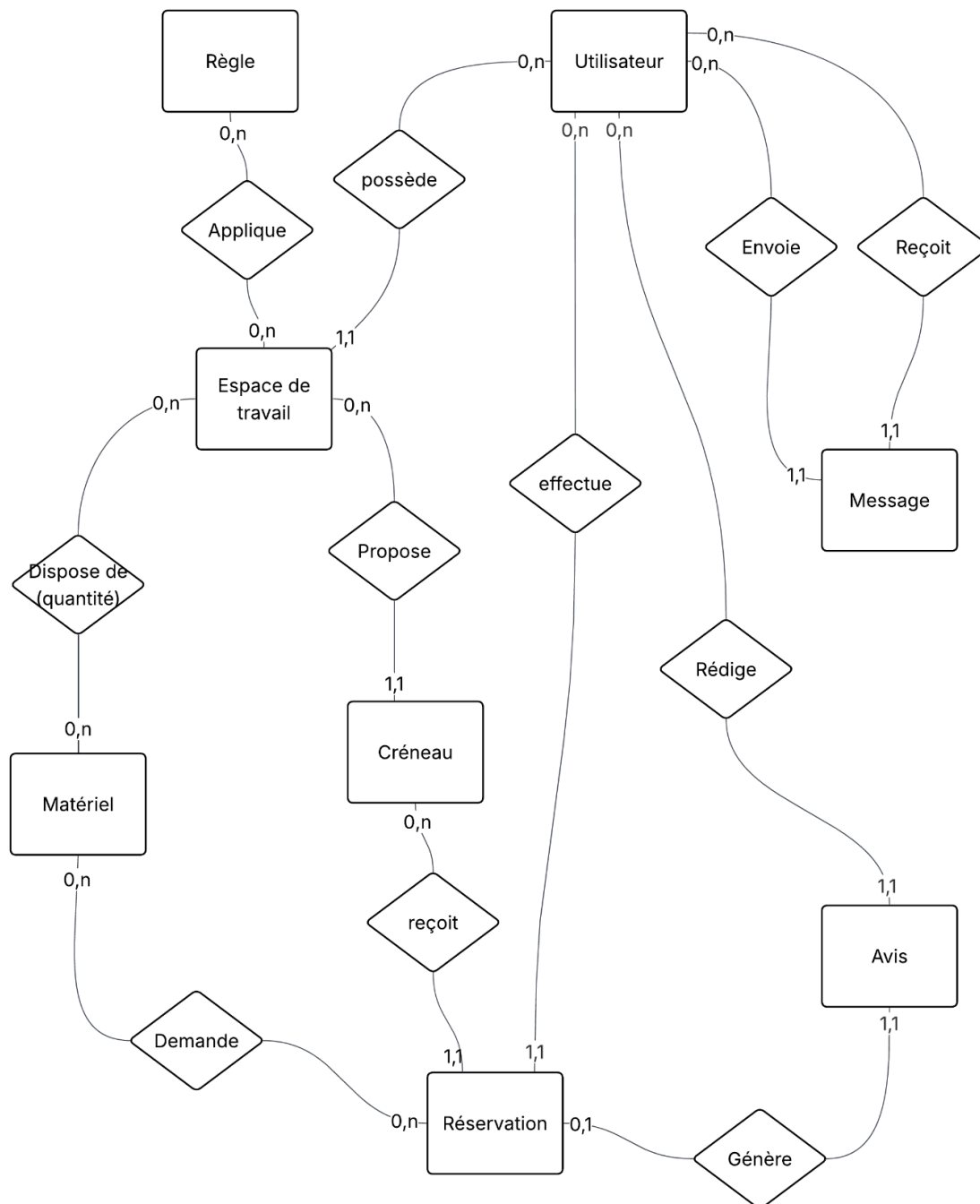


Figure 1 : Entité relation

7.2. Schéma relationnel

Le schéma relationnel, plus détaillé, permet d'avoir une vue complète de ce que devra être la base de données pour pouvoir implémenter la solution.

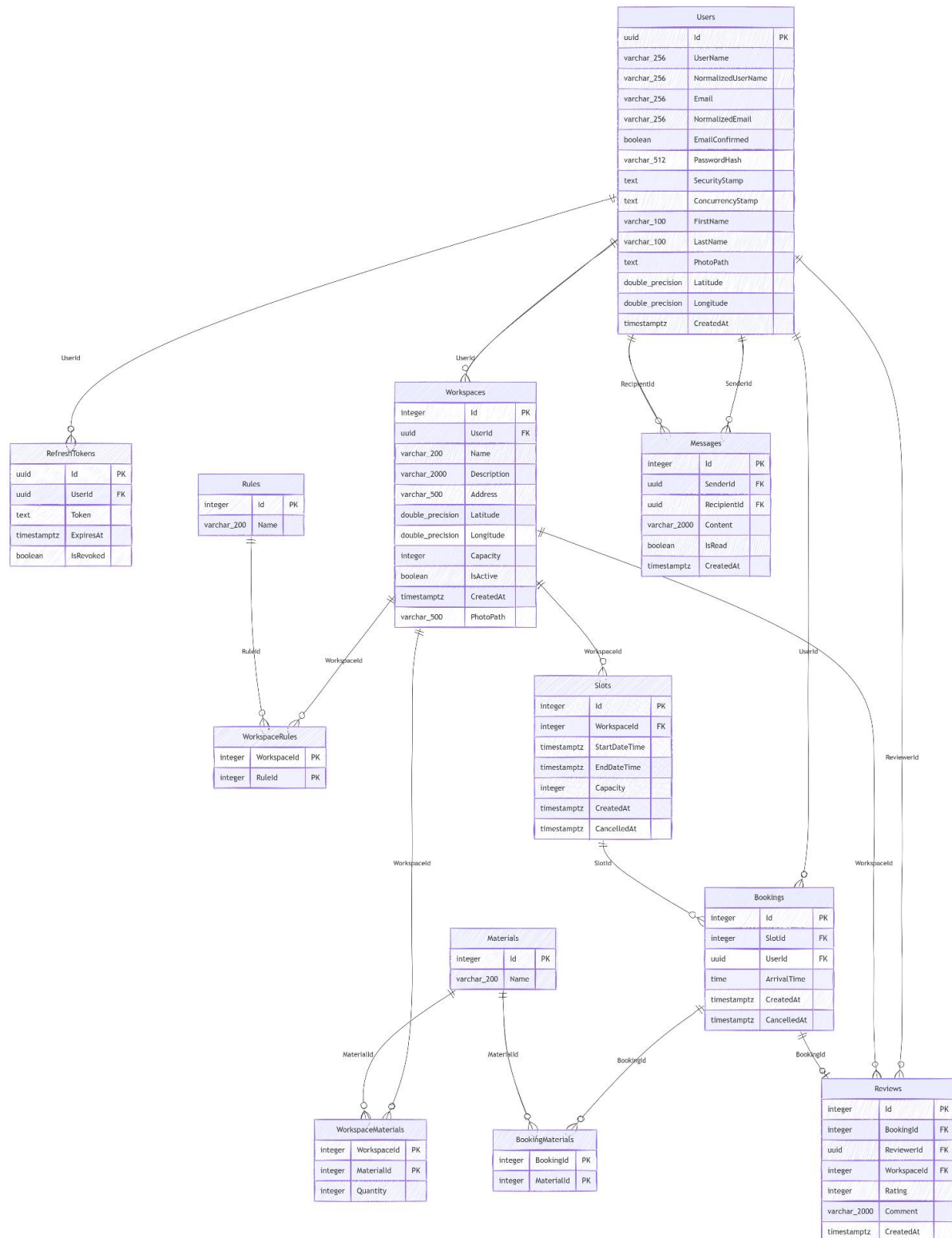


Figure 2 : Schéma relationnel

7.3. Entités

Dans ce chapitre, sont passées en revue, les différentes entités qui définiront dans le système. Nous avons 9 entités principales et deux entités de liaison. Ces dernières sont reconnaissables à leur nom qui est la concaténation des deux entités reliées.

7.3.1. Users

L'entité « User » ou « utilisateur » correspond à un utilisateur connu du système. Il est défini par un Guid unique permettant de le lier aux autres tables et de l'identifier, d'un nom, d'un prénom notamment. Du fait de l'héritage de .net Core Identity, d'autres champs sont présents comme le hash du mot de passe.

7.3.2. RefreshTokens

L'entité « RefreshToken » ou « Jeton de rafraîchissement » va contenir les jetons de rafraîchissement distribués par le backend lors d'une connexion par l'utilisateur. Quand le JWT sera expiré, le client enverra son jeton de rafraîchissement au backend pour réclamer un nouveau JWT valide. Plus d'informations seront disponibles dans le chapitre traitant de la sécurité.

La table contient un Id unique, une référence à l'utilisateur qu'il identifie, le jeton en lui-même, une date de péremption et un statut qui permet de le révoquer à tout moment.

7.3.3. Workspaces

L'entité « Workspace » ou « Espace de travail » correspond à l'espace de travail proposé par un utilisateur. Il est défini par un id unique, une référence à son auteur, une position géographique déterminée par une latitude et une longitude, un nom, une description et une adresse. Il a aussi un statut (booléen), un timestamp, un lien vers une photo optionnelle et une capacité.

Le modèle de donnée a été pensé pour pouvoir supporter plusieurs espaces de travail par hôte dans une prochaine version.

7.3.4. Slots

Un « slot » ou un « créneau » correspond au créneau mis à disposition par un hôte sur un espace de travail. Il contient un id unique, une référence à l'espace de travail, une capacité (qui peut être différente de celle de l'espace de travail), une heure de début et une heure de fin, un timestamp d'annulation nullable et un timestamp de création.

7.3.5. Bookings

Un « Booking » ou une « réservation » représente le souhait d'un utilisateur à prendre un créneau. Il contient donc un Id unique, une référence au slot et à l'utilisateur qui a fait la réservation, une heure d'arrivée, un timestamp d'annulation et un timestamp de création.

7.3.6. Messages

Un « Message » est une notification qui est générée à la suite d'une action faite par un utilisateur et est destinée à un autre utilisateur. Elle contient donc un id unique, une référence à l'utilisateur à l'origine du message, une référence à l'utilisateur auquel est destiné le message, un contenu ainsi qu'un état de lecture stocké sous la forme d'un timestamp et un autre de création.

7.3.7. Reviews

Les « reviews » sont les évaluations laissées par les utilisateurs. Une review porte sur une réservation et est donnée par un utilisateur. Une contrainte vérifie qu'un utilisateur ne puisse pas donner plusieurs fois son avis sur un même « workspace » pour éviter que les avis soient biaisés si une même personne est invitée de multiples fois chez le même hôte. Une évaluation est composée d'un id unique, d'une référence à une réservation, d'une référence à un auteur (nullable pour le cas où il serait effacé), d'une référence à un espace de travail, d'une note, d'un commentaire requis et d'une date de création.

7.3.8. Rules

Les « Rules » ou règles sont les règles qu'il est possible d'imposer dans un lieu de travail comme l'autorisation ou non d'apporter un animal de compagnie. Elles vont contenir un titre et un id unique.

7.3.9. WorkspaceRules

Les « WorkspaceRules » vont faire le lien entre les règles et les espaces de travail et vont déterminer quelles règles sont appliquées à quel espace de travail. Cette entité va uniquement contenir des références aux espaces de travail et aux règles.

7.3.10. Materials

Les « materials » représentent le matériel qui peut être proposé par les hôtes dans leur espace de travail. Ils contiennent une description. Comme les rules, ils ne contiennent qu'un nom et un Id unique.

7.3.11. WorkspaceMaterials

L'entité « WorkspaceMaterials » fait le lien entre l'espace de travail et le matériel. Il permet de déterminer le matériel disponible dans un espace de travail. Il contient donc une référence à ces 2 entités. Il contient aussi une quantité.

7.3.12. BookingMaterials

Les « BookingMaterials » représentent le matériel réservé par un invité dans l'espace d'un utilisateur. L'entité contient une référence à cet espace et au matériel.

8. Décisions technologiques

8.1. Détermination des supports

Le choix du ou des plateformes à supporter s'est posé très tôt dans le projet. Bien que WorkTogetherly puisse tout à fait s'utiliser depuis un ordinateur pour un travailleur qui planifierait sa semaine par exemple, il est également important de prendre en compte le fait qu'aujourd'hui beaucoup de gens utilisent aussi leur téléphone et ce marché ne peut être ignoré. D'autre part, dans le cadre d'un système de réservation, l'utilisation d'une version mobile va nous permettre d'intégrer des notifications natives dans une prochaine version par exemple et ainsi de tenir les utilisateurs au courant des réservations, annulations etc.

Par conséquent, il paraissait essentiel que WorkTogetherly soit disponible sur mobile comme en version web.

8.2. Choix de framework et langage

Etant donné que la décision avait été prise de proposer plusieurs supports avec un accès à des données centralisées, certains choix s'imposaient logiquement tels que la nécessité d'avoir une base de données partagée entre tous les supports.

Il y avait néanmoins une première décision primordiale à réaliser. De quelle manière, WorkTogetherly serait rendu dans le navigateur du client ? Il existe différentes manières de gérer la génération des pages dans un site web et il est important de les comprendre pour décider. Nous allons les passer en revue rapidement.

Le CSR ou Client Side Rendering propose la génération de l'html directement dans le client à l'aide de JavaScript. Un avantage très important est le fait de ne pas devoir recharger l'intégralité de la page lors de la mise à jour d'une partie de celle-ci. Ainsi, le serveur est généralement moins sollicité car il ne doit pas envoyer l'intégralité de la page HTML lors d'un changement. Cependant, il peut être plus difficile de gérer le référencement (SEO ou Search Engine Optimization) et peut être plus long au premier lancement car il implique d'éventuellement télécharger de nombreuses bibliothèques JavaScript. C'est cette solution qui est proposée par Vue, Angular, React et qui est très populaire de nos jours.

Dans le cas du SSR ou Server Side rendering, l'Html est directement généré depuis le serveur puis envoyé au client. Ici, il est alors tout de même toujours possible d'utiliser du Javascript pour gérer l'interactivité. L'avantage est que le site est plus rapidement chargé au premier lancement par le client mais il peut paraître plus lent lors de la navigation entre les pages en raison du rechargement complet de la page. Les solutions telles que

PHP, Razor pages ou de solutions plus récentes comme Next.js en sont quelques exemples.

Nous trouvons aussi le SSG ou static site generation. Les générateurs de sites statiques proposent de générer des sites statiques qui seront visibles de la même manière pour tous les clients pour une même url. Les pages sont construites lors de la génération du site et non lors de la consultation comme ce serait le cas en SSR. Des solutions telles que Astro, Jekyll ou encore VitePress proposent ces solutions. Ils sont rapides, sécurisés et faciles à déployer par leur absence de logique côté serveur.

Enfin, un autre concept que nous allons aborder est le WebAssembly ou WASM . Wasm est un type de code à destination des navigateurs web modernes. C'est un langage de bas niveau permettant d'avoir d'excellentes performances et permettant d'utiliser des langages tels que C#, C ou C++ pour générer le binaire. Lors du premier chargement du site, le binaire devra être téléchargé depuis le serveur puis le code sera exécuté intégralement sur le navigateur du client. C'est une technologie relativement récente, publiée en 2017 mais qui a de grandes implications sur le web car elle est assez novatrice. En revanche, là aussi, le référencement peut être plus compliqué. Des solutions comme Rust ou Blazor WebAssembly proposent de faire tourner du WASM dans le navigateur.

En plus de ces catégories, il existe de nombreuses solutions entre les deux, des hybrides reprenant les forces des uns et des autres pour les combiner et tenter de gommer les défauts et potentiellement d'autres solutions moins connues et moins supportées.

Dans le cadre du projet, il était important de choisir une technologie qui permette de créer facilement une application mobile et par conséquent, de choisir une technologie qui puisse partager le même backend que l'application web et le même type d'authentification mais surtout une technologie qui permette de générer les clients pour les différents supports à partir d'un code source commun. Ceci permettrait d'éviter de développer plusieurs fois le même code source dans différents langages.

De plus en plus de frameworks proposent le partage de code entre mobile et site web.

En termes de front end, plusieurs solutions permettent de créer une application mobile et un site web facilement : React Native et React, Flutter, Ionic + Capacitor, Blazor, ...

Concernant le backend, beaucoup de solutions existent telles que Php (Laravel, CakePhp, symfony), .net, les bibliothèques JS pour serveur (Node.js,...), Java, Django (python), ...

Blazor propose une solution permettant d'utiliser un même langage en backend et en front end de manière unifiée dans un seul et même langage. Il était intéressant d'analyser les différents modes de rendu qu'il proposait :

- Rendu statique serveur : Les pages demandées sont générées et servies au front end par le back end sans aucune interactivité (pas d'ouverture de WebSocket pour maintenir la communication entre le front end et le backend)
- Rendu interactif serveur : Il s'agit du même mode que le précédent à la différence que le backend et le front end maintiennent une connexion permanente entre les deux et permettent de l'interactivité qui n'est pas possible avec le rendu statique serveur.
- WebAssembly (WASM) : exécution de code C# sur le navigateur.
- Auto interactif : qui va démarrer en mode rendu interactif serveur puis passer en WebAssembly lorsque les données seront téléchargées.

En termes de Mobile, c'est MAUI qui se charge de créer une application mobile. L'application contient un BlazorWebView qui permet de rendre les composants blazors dans le webviewer natif de l'appareil en les convertissant en HTML. Ceci permettrait de réutiliser les composants entre les différentes solutions.

La conclusion était que Blazor offrait une solution intéressante proposant, à partir de C# uniquement, de créer une application mobile, un site web et le backend correspondant. La solution existe depuis plusieurs années, preuve d'une certaine maturité, et utilise des techniques résolument modernes (avec le WASM). Le mode auto interactif paraissait particulièrement séduisant (offrant le meilleur des 2 mondes, SSR et Wasm).

Lors de l'utilisation du mode SSR, puisque l'application tourne sur le serveur et pas dans l'appareil du client, il est possible de faire une application qui ne nécessiterait pas d'API puisque le serveur qui génère les pages a un accès direct aux données. Cependant, dans le mode de fonctionnement WebAssembly, puisque l'application tourne directement dans le navigateur du client, il faut mettre en place des requêtes Http depuis la machine du client vers les serveurs pour créer, lire, mettre à jour ou effacer des données. De même, dans le cas de MAUI, les données n'étant pas directement stockées sur le mobile de l'utilisateur, il faudra là aussi passer par l'étape de mise en place des APIS.

En utilisant le mode auto, il y aurait donc 3 fronts ends différents à gérer et la décision d'unifier le code en passant par des APIS dans tous les cas pour éviter de complexifier le code fut prise. Dans le cas du SSR, le serveur ferait des appels API vers lui-même. Les avantages de cette méthode sont donc d'éviter d'accroître la complexité mais aussi de permettre d'héberger plus facilement le backend et le front end sur des serveurs différents au prix d'une latence un peu plus élevée dans le mode SSR.

Après quelques essais, ce choix fut quelque peu simplifié et le support du SSR fut abandonné dans cette version. La raison de cette décision est un processus d'authentification trop compliqué. Plus de détails seront disponibles dans le chapitre dédié à la sécurité mais passer les JWT (Json Web Tokens) et les « Refresh tokens » de l'application Web à celle en WebAssembly lors du passage automatique entre les deux

aurait engendré une difficulté considérable dont la résolution aurait demandé de dépasser le cadre de ce projet. Wasm et MAUI fonctionnant tous les deux sur le client, ce choix avait d'autant plus de sens pour une première version.

Le prix de cet abandon est un temps de lancement de l'application un peu plus long et un moins bon référencement. Ce dernier point n'est pas vraiment un problème pour un outil tel que WorkTogetherly mais il faudrait prévoir un site statique pour l'accompagner et faire sa promotion ou revenir sur ce point dans une prochaine version.

8.3. Backend

Etant donné que le backend serait développé séparément du front end, il serait fait en .net et n'aurait rien à voir avec Blazor. Pour mettre en place un back end propre, maintenable et qui puisse évoluer sur le long terme, une série de bonnes pratiques ont été mises en place et seront détaillées dans ce chapitre.

8.3.1. La clean Architecture

Le choix d'une architecture backend est impérative afin d'éviter que le code devienne rapidement ingérable. Deux grandes approches ont été envisagée. La première, la division en couche (repository, service, contrôleur) qui est souvent pratiquée aurait apporté une simplicité de mise en place au prix d'une dépendance plus forte de la logique métier à l'infrastructure. La seconde, la clean architecture, apporte une meilleure scalabilité et testabilité au prix d'une plus grande complexité à la mise en place et une courbe d'apprentissage un peu plus élevée.

Il était intéressant d'essayer la clean architecture dans le cadre de ce projet et de profiter de cette occasion pour mieux comprendre cette architecture. C'était quelque chose de totalement nouveau et qui a demandé un certain temps d'adaptation.

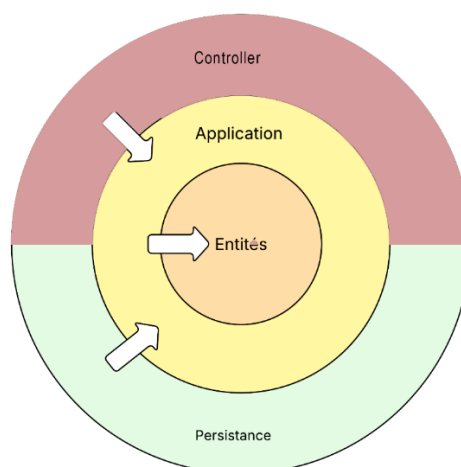


Figure 3 : représentation de la clean Architecture

La clean architecture divise l'application en 4 couches :

- Le domaine (les entités) : Nous y trouvons les objets du domaine métier c'est à dire les utilisateurs, les réservations, etc. Contrairement à l'architecture n-tiers, on n'y trouve aucune de notion de base de données ou de persistance des données. On y trouve cependant la logique métier liée aux entités. On définit aussi les interfaces dans cette couche : le principe est de définir ce dont on a besoin pour les entités mais pas la manière de le faire. Cette couche, centrale, peut être utilisée par toutes les couches supérieures et doit avoir le moins de dépendances possibles.
- L'application : Elle contient les uses cases. Elle reçoit les demandes de la couche de présentation et les orchestrent.
- La présentation : Il s'agit des contrôleurs qui ont la charge de réceptionner les requêtes http provenant de l'extérieur et de les rediriger vers la méthode requise de l'application et de répondre à ces requêtes par le biais de codes (200, 201, 500 etc.) en incluant un détail quand nécessaire.
- L'infrastructure : Elle est responsable de la sauvegarde et de la récupération des données en implémentant les interfaces définies dans le domaine. Elle utilise Entity Framework ou l'ORM choisi. Elle est aussi responsable de l'implémentation de toutes les interfaces permettant de connecter l'application avec des services externes (envoi de courrier électronique par exemple).

8.3.2. CQRS

CQRS est l'abréviation de Command Query Responsibility Segregation (ou Ségrégation des Responsabilités en Commande et Requête). Il s'agit d'un modèle architectural qui repose sur une division claire entre les commandes (opérations de modification des données) et les requêtes (opérations de récupérations de données).

D'un côté, les commandes modifient l'état du système en ne retournant que très peu d'informations. De l'autre côté, les requêtes récupèrent l'état du système sous forme de DTO sans le modifier et peuvent passer totalement le domaine pour aller directement en base de données.

Grâce à cette séparation, CQRS présente l'avantage de pouvoir optimiser les opérations de lecture et d'écriture de la base de données dont les différences ne sont pas toujours bien prises en compte par les autres architectures.

Chaque commande ou requête possède un handler dédié qui sait comment le traiter et doit le faire de manière autonome. Le rôle du handler doit être limité à un orchestrateur du cas d'utilisation et on doit trouver un handler (et donc une classe) par cas d'utilisation.

Il est important de comprendre que CQRS n'est pas obligatoire dans la clean architecture et que CQRS peut être utilisé hors de la clean architecture. Par conséquent, bien que souvent utilisés conjointement, ces deux concepts sont indépendants. Aussi, il est important de comprendre que CQRS est généralement conseillé dans des applications dont l'écriture et la lecture présentent une forte asymétrie ou dans les applications traitant un nombre important de données et où CQRS permettra de garantir de meilleures performances. On conseillera aussi CQRS quand plusieurs équipes de développeurs travaillent en parallèle sur un même produit et dans le cas où une équipe pourrait travailler sur les écritures quand une autre s'occuperait des lectures.

Comme toute solution, CQRS apporte son lot de désavantages comme une complexité accrue (il est parfois difficile de s'y retrouver) ou une incohérence entraînée par des écritures qui peuvent se faire indépendamment des lectures (et donc être concurrentes). De manière générale, on peut se passer de CQRS quand les règles métiers sont simples ou ne nécessitent que des opérations CRUD (Create, Read, Update, Delete).

WorkTogetherly n'aurait probablement pas nécessité CQRS si les 3 raisons que voici n'avaient pas justifié la décision de l'implémenter :

- Beaucoup d'exemples de Clean architecture utilisent CQRS et ils sont souvent associés. Il était donc plus facile de trouver des réponses aux questions en passant par ce modèle.
- CQRS permet de facilement intégrer de manière transparente FluentValidation dans le processus de validation des données (à voir dans le chapitre dédié).
- Il aurait été dommage de terminer ce cursus sans avoir eu la chance d'expérimenter ce modèle architectural.

La bibliothèque utilisée pour le mettre en place a été la plus connue : MediatR. Ce produit existe dans une version gratuite limitée à des fins de tests. Il est récemment passé à un modèle payant pour une utilisation commerciale.

8.3.3. Les Domain Events et le système de notification

Dans le cadre de l'analyse du projet, il paraissait essentiel de mettre en place un système de notification pour alerter les invités lors de la modification ou l'annulation d'un créneau qu'ils ont réservé. De la même manière, pour un hôte, il est important d'être informé quand un utilisateur effectue, modifie ou annule une réservation.

S'il avait fallu le mettre en place en Laravel, que j'utilise dans le milieu professionnel, il aurait fallu procéder à la mise en place d'« observer » afin de surveiller les modifications apportées aux entités de réservations et de créneaux et d'appliquer une action quand celle-ci se produit. Le but serait d'éviter qu'un autre développeur ajoute une nouvelle fonctionnalité sans prendre en compte le besoin de notification.

Dans le cadre de ce travail, grâce à la mise en place de la Clean architecture, le principe de Domain Event qui est lui-même un des principes du DDD, Domain Driver Design, a pu être utilisé. Il s'agit d'émettre un événement directement depuis le domaine lors de la modification de l'entité dans les méthodes d'annulation, de création ou de modification. Ces événements pourront ensuite être capturés depuis la couche application qui pourra orchestrer la logique nécessaire au cas d'utilisation : l'envoi d'une notification sous quelque forme qu'elle soit. La logique d'envoi sera gérée par l'infrastructure.

Pour mettre cela en pratique, MediatR a aussi été utilisé. Cela a cependant pour conséquence d'ajouter une dépendance du domaine à un package externe, dépendance qui va à l'encontre du conseil de garder le domaine le plus libre de dépendance possible. Cela semble un compromis acceptable si on considère que l'implémentation des événements se fait en dehors de la couche centrale du domaine.

8.3.4. La gestion des erreurs métiers et le Result Pattern

Dans le cadre de ce projet, il fallait éviter que les exceptions d'erreurs métiers soient jetées de n'importe quel endroit du code ou éviter les soucis traditionnels d'une méthode qui renverrait « null » ou « false » quand une entité n'est pas trouvée par exemple. Sans de bonnes pratiques, la gestion des erreurs métiers devient complexe à gérer. Le principe derrière le « result pattern » est qu'une méthode doit toujours renvoyer un objet contenant la réponse business (un DTO généralement) et les erreurs éventuelles sous un même format (représenté par un objet). Le Controller devra recevoir ces objets de la couche applicative et simplement envoyer le code d'erreur qui conviendra à la situation en réponse à la requête HTTP.

Cela a aussi l'avantage de centraliser toutes les erreurs métiers dans des classes et fichiers séparés qui permettent très facilement de les modifier ou de les tracer.

Il existe différentes alternatives de librairies pour faire cela : ErrorOr, OneOf, FluentResult,... ErrorOr était simple à mettre en place et répondait parfaitement au besoin.

Pour éviter de devoir convertir le même type d'erreur en même code d'erreur dans chaque contrôleur, il a fallu mettre en place une classe, ApiController, qui permettrait de faire les conversions et dont hériterait tous les contrôleurs pour ainsi éviter toute redondance inutile de code.

Un point d'attention est à porter sur le fait qu'étant donné que cette bibliothèque est utilisée dans la couche domaine, elle ajoute une autre dépendance au domaine mais que, après réflexion, ce compromis était acceptable car il ajoute aussi une grosse plus-value par sa clarté.

8.3.5. La validation des entrées et FluentValidation

Afin d'éviter de surcharger l'application et les entités avec de nombreuses règles de validation, l'utilisation d'une bibliothèque .net populaire fut prise pour se charger de cette responsabilité : FluentValidation. Celle-ci permet de définir des règles de validation pour chaque type dans des classes de validation dédiées.

Ces validations sont placées dans la couche « Application » car elles constituent des validations métiers et pas des validations techniques. Grâce au package MediatR, il est possible de définir un pipeline de validation permettant d'effectuer automatiquement la validation des données entrantes dans les handlers et de jeter une erreur de type « validationError » si une règle n'est pas respectée. Ce mode de fonctionnement est très utile pour séparer la validation des entrées de la logique métier.

8.3.6. La base de données

WorkTogetherly manipule des données fortement structurées. Un utilisateur gère un espace, qui comporte des créneaux. Ces créneaux sont réservés et génèrent des avis, eux-mêmes liés à des invités. Ces relations multiples couplées aux contraintes d'intégrités strictes (on ne peut pas réserver son propre espace, un créneau ne peut pas être surréservé, ...) ont naturellement orienté le choix vers une base de données relationnelle plutôt que vers un modèle tel que NoSql plus adapté à des données non structurées. D'autre part, l'aspect collaboratif implique une base de données centralisée et partagée : plusieurs utilisateurs sur plusieurs appareils doivent accéder aux mêmes données en temps réel. Une base de données interne au navigateur ou à l'application mobile serait totalement inadaptée.

Concernant l'ORM (Object-Relational Mapping) servant d'intermédiaire entre le langage de programmation et la base de données, 2 alternatives principales se présentent :

- Dapper : Très performant, ressemble fortement à du RawSql avec un contrôle précis sur les requêtes qui sont exécutées par le serveur.
- EntityFramework : Parfois moins performant et plus abstrait mais plus rapide à utiliser, gère les migrations facilement.

Il aurait aussi été possible d'utiliser les deux conjointement : un pour les requêtes, l'autre pour les commandes. Etant donné que le besoin de gérer finement les requêtes ne se faisait pas spécialement sentir dans le contexte de ce projet, l'utilisation d'Entity Framework semblait une bonne idée. De plus, il permettrait de générer facilement la base de données en « code first », c'est-à-dire en créant les entités dans le code et en exécutant ensuite une migration pour générer la base de données.

Enfin, le dernier choix à faire était de choisir le fournisseur de la base de données à proprement parler. Etant donné que la base de données serait située derrière l'ORM, ce choix aurait moins d'importance pour peu qu'il soit supporté par EntityFramework. Là aussi, il y a beaucoup d'alternatives : PostgreSQL, MySQL, Oracle, SQLite, MariaDB ou encore Microsoft SQL (MS SQL). De plus, la plupart semblent supporter .net 10 (même si la documentation sur ce sujet ne semble pas très à jour).

Les premières versions du projet ont été construites avec MS SQL, ce qui semblait correspondre au reste du projet qui était entièrement créé dans l'écosystème Microsoft.

Cependant, plus tard, le choix a été fait de migrer vers une base de données PostgreSQL car le fait que ce soit gratuit permettait de supporter des solutions moins onéreuses pour le déploiement (Plus de détails dans le chapitre dédié au déploiement).

8.3.7. Les tests unitaires, Xunit et Mock

Dans le but d'éviter toute régression lors du développement du projet, le choix a été fait d'implémenter des tests unitaires dans le backend dès que les premières fonctionnalités seraient mises en place.

Pour garder le code clair un projet séparé au sein de la solution a été mis en place pour séparer les tests du reste du code. A l'intérieur de ce projet, tous les tests seraient classés par couche.

Xunit, l'outil de test, est directement intégré au sein de VisualStudio et permet de tester le backend en quelques clics. En complément, Mock permet de facilement moquer les objets qui devraient normalement être injectés dans le constructeur de la classe testée. Il est ainsi possible de tester unitairement chacune des couches, des classes et des méthodes indépendamment des autres.

Une mention particulière pour la couche infrastructure qui est testée à l'aide d'une image docker « ensemencée » automatiquement. De cette manière, elle peut être testée autant de fois que nécessaire sans devoir dépendre d'une véritable base de données.

8.3.8. Les websockets et SignalR

Il peut parfois être nécessaire d'établir un canal de communication bidirectionnel entre le serveur et le client pour informer le client d'un changement, de la fin d'une tâche etc. C'est le type de fonctionnalité offert par SignalR en .net.

SignalR établit une connexion par websocket s'il est supporté mais revient à des méthodes de transports plus anciennes s'il n'est pas supporté par une des parties.

Le websocket est une solution bien plus efficace que ce qui pourrait être fait via un système de polling par exemple (en polling, le front demande toutes les 2 secondes si un nouveau message est disponible). Il s'agit aussi d'une solution présentant des latences minimales.

Dans le contexte de WorkTogetherly, lorsqu'un message est envoyé vers un invité ou vers un hôte, on désire que l'utilisateur reçoive immédiatement l'information dans son navigateur ou dans son application mobile. C'est à cet endroit que la mise en place de cette fonctionnalité a pris tout son sens.

8.4. Front End

8.4.1. Langage et Architecture frontend

Comme nous l'avons indiqué précédemment, un même code va être partagé entre les différentes plateformes.

Le projet front end est donc sous-divisé en 3 projets permettant de rendre la solution disponible sur les différentes plateformes avec un minimum de répétition de code et ainsi d'éviter les différences de comportement entre les solutions.

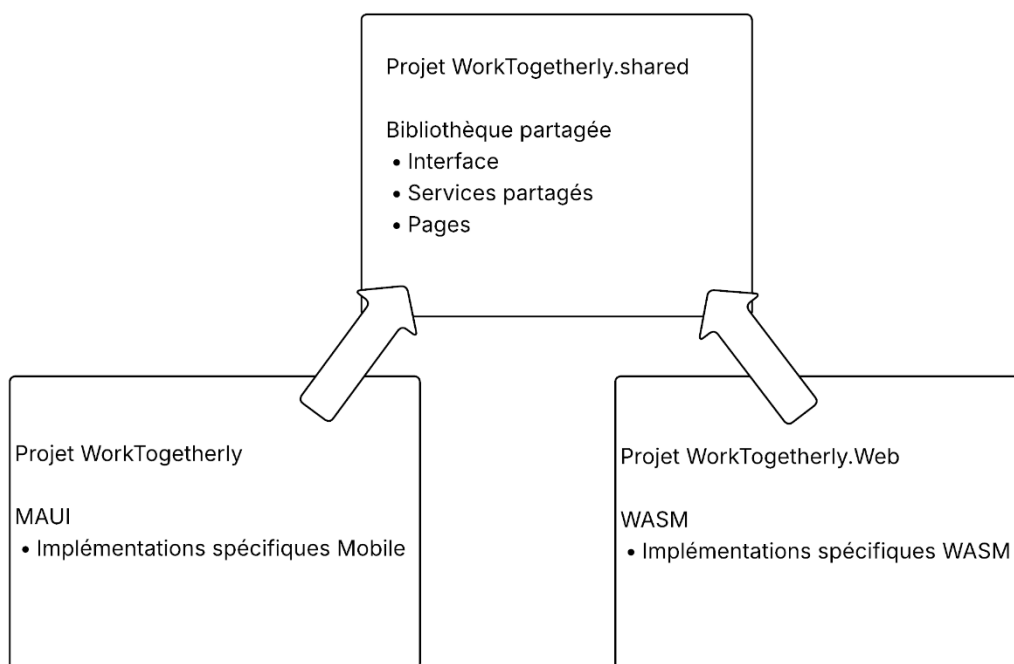


Figure 4 : Structure du front end

8.4.1.1. *Le projet partagé*

Le projet partagé, WorkTogetherly.Shared va contenir tout ce qui sera partagé entre les différents projets. Il contiendra tous les pages qui seront affichées sur les différentes plateformes, les mises en pages, les modèles des réponses, les services partagés ou encore les interfaces de services qui devront être implémentés dans les autres projets pour fonctionner correctement.

8.4.1.2. *Le projet MAUI*

Le dossier WebTogetherly va permettre de définir les spécificités pour le fonctionnement de l'application (Android, ios ou windows) en permettant de définir du code pour chaque système d'exploitation différent ou pour les applications mobiles de manière générale. Par exemple, c'est dans ce dossier que nous avons défini des règles spécifiques pour le stockage des tokens ou la géolocalisation.

8.4.1.3. *Le projet Web.Client*

Ce projet va permettre de définir du code spécifique pour le code en Web Assembly. Une fois que l'application est chargée en WebAssembly, elle tourne entièrement dans le navigateur et ne fera plus que des appels au backend que pour récupérer ou pousser des données par exemple. Ceci lui donne un avantage de rapidité si la machine de l'utilisateur le permet.

8.4.2. La bibliothèque MudBlazor

En termes de bibliothèque de composant Front End pour Blazor, différentes alternatives sont disponibles :

- MudBlazor
- Blazor UI
- Blazorise
- Radzen
- FluentUI

Il est aussi possible de se diriger vers du Bootstrap simple mais les outils précédemment cités proposent des mécanismes propres à Blazor. Mudblazor proposait une bibliothèque bien documentée, proposant de nombreux outils facilement prenables en main. Après avoir testé rapidement Bootstrap seul qui demandait trop de modifications pour obtenir un résultat satisfaisant, MudBlazor a apporté une solution satisfaisante.

C'est-à-dire une solution simple avec des composants pré-faits, demandant un temps d'adaptation réduit tout en étant utilisable sur mobile comme sur écran large.

Le thème par défaut, principalement mauve, ne convenait pas vraiment à WorkTogetherly. C'est pourquoi un thème personnalisé a été mis en place avec des couleurs plus chaudes qui correspondrait mieux aux besoins.

8.5. Sécurité

La communication entre les fronts end et le backend se fait principalement via des API REST. Etant donné que nous avons une application mobile et une application de type WebAssembly, il est important de trouver un terrain commun pour que les 2 puissent dialoguer de manière sécurisée avec le backend. En prenant en compte le fait qu'il y ait une application mobile, Il paraissait d'autant plus important de proposer une application mobile qui permette aux utilisateurs de rester connecté pendant plusieurs jours et ainsi éviter la frustration de devoir entrer ses identifiants fréquemment.

La solution la plus répandue est la mise en place d'un JWT accompagné d'un jeton de rafraîchissement. Le principe est le suivant : lors de l'authentification, l'utilisateur reçoit un JWT (plus rapide, moins sécurisé) qui périmé rapidement et un jeton de rafraîchissement bien plus long (plus lourd, longue durée de vie, plus sécurisé).

Le JWT ou Json Web Token est un jeton généré par le backend répondant aux normes RFC7519. Il contient directement toutes les informations importantes au sein de sa charge utile. Il est composé de 3 parties :

- Un header : le header ou en-tête va contenir le type de jeton et des informations sur la manière dont sa signature a été générée. Exemple : `{"alg": "HS256", "typ": "JWT"}`.
- Un Payload : Le payload ou la charge utile va permettre de stocker différentes informations qui vont nous permettre d'identifier l'utilisateur connecté et de transférer certaines de ses informations au récepteur du JWT. Exemple : `{ "sub": "123", "name": "Alice", "exp": 30 }`
- Une signature : La signature va être générée en prenant la base 64 du header et du payload et en le signant via une clef secrète. En général, cette clef est détenue par le backend pour ne pas être rendue publique.

Le JWT est donc un jeton lisible par tout le monde mais seul le backend peut en générer et peut vérifier l'authenticité de l'un d'entre eux. Un JWT n'a pas de date de péremption et, par son état « stateless », il n'est pas stocké dans une base de données ce qui rend sa révocation difficile. C'est pour cette raison que nous lui donnons une durée de vie très courte. Pour le WebAssembly comme pour l'application, le JWT va être stocké temporairement.

Pour compléter l'implémentation, nous allons procéder à l'ajout d'un « refresh token » ou « jeton de rafraîchissement ». Ce dernier est un jeton à la durée de vie bien plus longue. Il est généré avec des caractères pseudos-aléatoires et sera stocké en base de données. Lorsqu'un JWT est attribué à un utilisateur lors de sa première connexion, le front end reçoit aussi ce jeton qui sera stocké :

- Dans les cookies pour l'application WebAssembly.
- Dans le Stockage sécurisé du système d'exploitation pour l'application mobile.

Chaque fois qu'une requête recevra une réponse « 401 », les services de gestion de l'authentification vont tenter d'utiliser le jeton de rafraîchissement pour obtenir un nouveau JWT et rejouer la requête qui avait été bloquée par le backend. Un Endpoint dédié du backend peut vérifier la validité d'un jeton de rafraîchissement et déterminer à quel utilisateur il appartient. Le jeton de rafraîchissement n'est utilisé que pour recevoir un nouveau JWT et jamais pour authentifier une requête directement.

Les annexes comprennent quelques schémas des flux mis en place pour cette gestion.

8.6. Méthodologie de travail

Bien qu'il ait été envisagé de travailler suivant une méthodologie Agile ou Shape Up comme c'est souvent fait en entreprise, la simplicité a été privilégiée puisqu'il n'y avait pas de nécessité de synchroniser le travail entre plusieurs développeurs.

Un simple projet Kanban a été suffisant pour créer les tâches principales du projet. Par la suite, les issues ont été ajoutées au fur et à mesure que des bugs ou des améliorations étaient requises.

L'idée de mettre en place une compilation automatique dans GitHub a été envisagée mais a par la suite été écartée car non supportée à cause de MAUI.

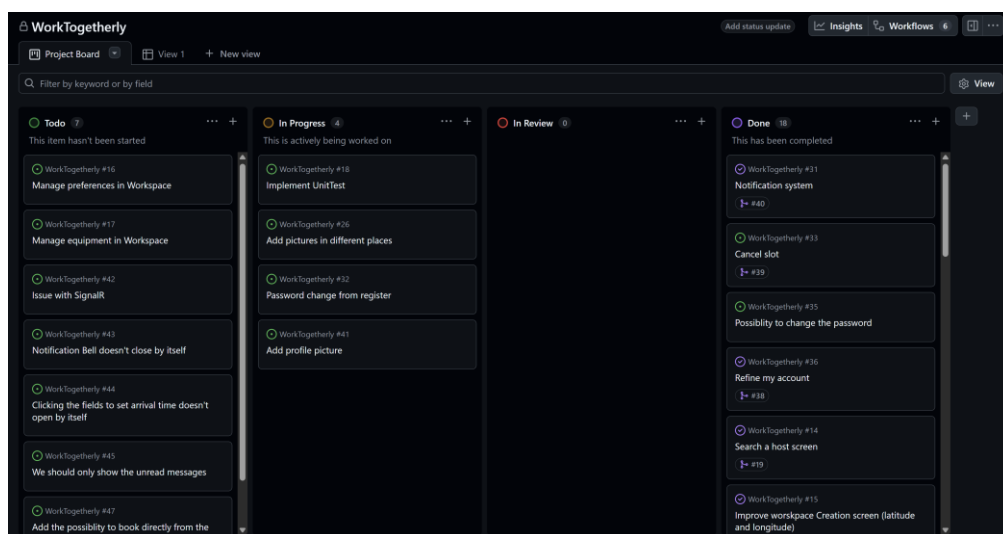


Figure 5 : Kanban du projet

Concernant la gestion des branches, GitFlow aurait été choisi si l'équipe avait été plus grande (avec master, develop, release). Dans ce cas-ci, la branche principale a été le master. Les identifiants des tâches ont été utilisés dans les « commits » et dans les branches autant que possible. Cela permettait d'identifier facilement à quoi pouvait être dû une régression quand elle se présentait comme le veut la bonne pratique.

Lors de la réalisation du projet, une des premières étapes a été la mise en place d'une maquette simple, mettant en place les différents éléments nécessaires par la suite c'est-à-dire une structure générale pour le projet, le développement d'une page simple de login et de création de compte, comprendre le fonctionnement de MAUI, WASM, les composants MudBlazor etc... Une fois passées les premières difficultés et avoir vérifié que les choix technologiques étaient viables, la priorité est revenue aux analyses du cahier des charges. Le développement réel serait entrepris bien plus tard.

Ce mode de fonctionnement a été payant lors de la réalisation du projet.

8.7. Déploiement

Lors de la mise en place du cahier des charges, il est rapidement devenu évident que le fait de vouloir une application mobile aurait des impacts non négligeables sur les développements. Au moment de tester l'application sur un émulateur, l'idée que le front-end cherchant le backend sur une adresse localhost allait causer des soucis s'est imposée. Il était probablement possible de contourner ce problème mais déployer l'application serait une expérience instructive. De plus, l'expérience sur smartphone serait bien meilleure en déployant le backend sur une URL publique et ainsi faire tester l'application par certains beta testeurs. Cela permettrait de récolter des avis, des suggestions ou des bugs qui pourraient être corrigés.

Certains tests ont été menés avec la plateforme bien connue de Azure mais malheureusement, cette solution serait rapidement devenue couteuse à mettre en place. En effet, la mise en place d'une base de données n'est pas compris dans l'offre de base.

L'alternative choisie a été Railway qui propose de mettre en ligne une application de manière assez simple et pour une somme modique. Elle propose notamment le déploiement automatique d'une branche GitHub dès qu'un changement est poussé dans celle-ci. Deux changements ont été nécessaires pour faire fonctionner correctement l'application dans la plateforme Railway

- La conteneurisation de l'application dans Docker pour qu'elle puisse être plus facilement déployée.
- Le passage de MS SQL à PostgreSQL, le premier des deux étant une solution propriétaire.

Comme prévu, ce déploiement fut une expérience enrichissante et a grandement facilité l'utilisation et la réalisation des tests sur mobile.

9. Considérations budgétaires

Il peut être complexe d'évaluer les coûts exacts de la plateforme sans connaître le nombre d'utilisateurs et la charge que devra supporter l'application. Néanmoins, l'expérience du déploiement sur Railway peut nous illustrer les coûts minimums de fonctionnement. Pour pouvoir fonctionner, l'application nécessitera au minimum un abonnement chez un hébergeur (Railway utilisé actuellement dans la solution propose une version limitée à 5 euros par mois), un abonnement éventuel auprès d'un fournisseur d'email (Resend qui est utilisé dans la solution est gratuit pour 3000 emails par mois), un nom de domaine (une dizaine d'euros par an) et surtout une licence pour l'utilisation de MediatR (50 euros /mois pour la production).

Pour les stores Android et Apple, il faudra là aussi payer une licence autorisant la publication.

Si on prévoit des notifications push à l'avenir, il faudra aussi compter un abonnement pour ce service.

10. Vue et écrans

Assez tôt, la meilleure manière d'organiser les écrans a été une des priorités. En particulier, la solution devant être utilisable sur mobile, il fallait éviter les menus longs et fastidieux et prévoir un voyage le plus naturel possible pour les utilisateurs et impliquant un minimum de clics.

Généralement, on trouve 2 grands types de navigation :

- La barre latérale sur le côté gauche (qui se ferme automatiquement sur mobile).
- La barre de navigation positionnée en bas de l'écran.

La direction prise fut d'utiliser la première sur ordinateur et la seconde sur mobile. En effet, la première est ce que nous trouvons généralement lors de la navigation sur un ordinateur tandis que la deuxième est pratique sur mobile car toujours présente sans nécessité de clic pour ouvrir la barre latérale. Cependant, elle demande de ne pas avoir plus de 4 ou 5 options disponibles pour ne pas paraître surchargée.

La contrainte était par conséquent, une fois l'utilisateur connecté, de limiter au maximum les vues disponibles. Les fonctionnalités ont été divisées en 4 écrans principaux depuis lesquels il serait possible de faire toutes les actions grâce à des fenêtres modales. Voici ces 4 écrans :

- Ecran de gestion en tant qu'hôte qui permettrait de créer, modifier ou annuler son espace, ses créneaux, visualiser les réservations des invités, les avis, etc.
- Ecran de gestion en tant qu'invité qui permettrait de voir ses réservations passées ou futures, de les modifier, de laisser des avis, etc.
- Ecran de configuration du compte : permettant de modifier ses informations de base, mot de passe, etc.
- Ecran de recherche de créneau : Filtres, vue en liste ou en carte pour consulter les espaces de travail et faire des réservations.

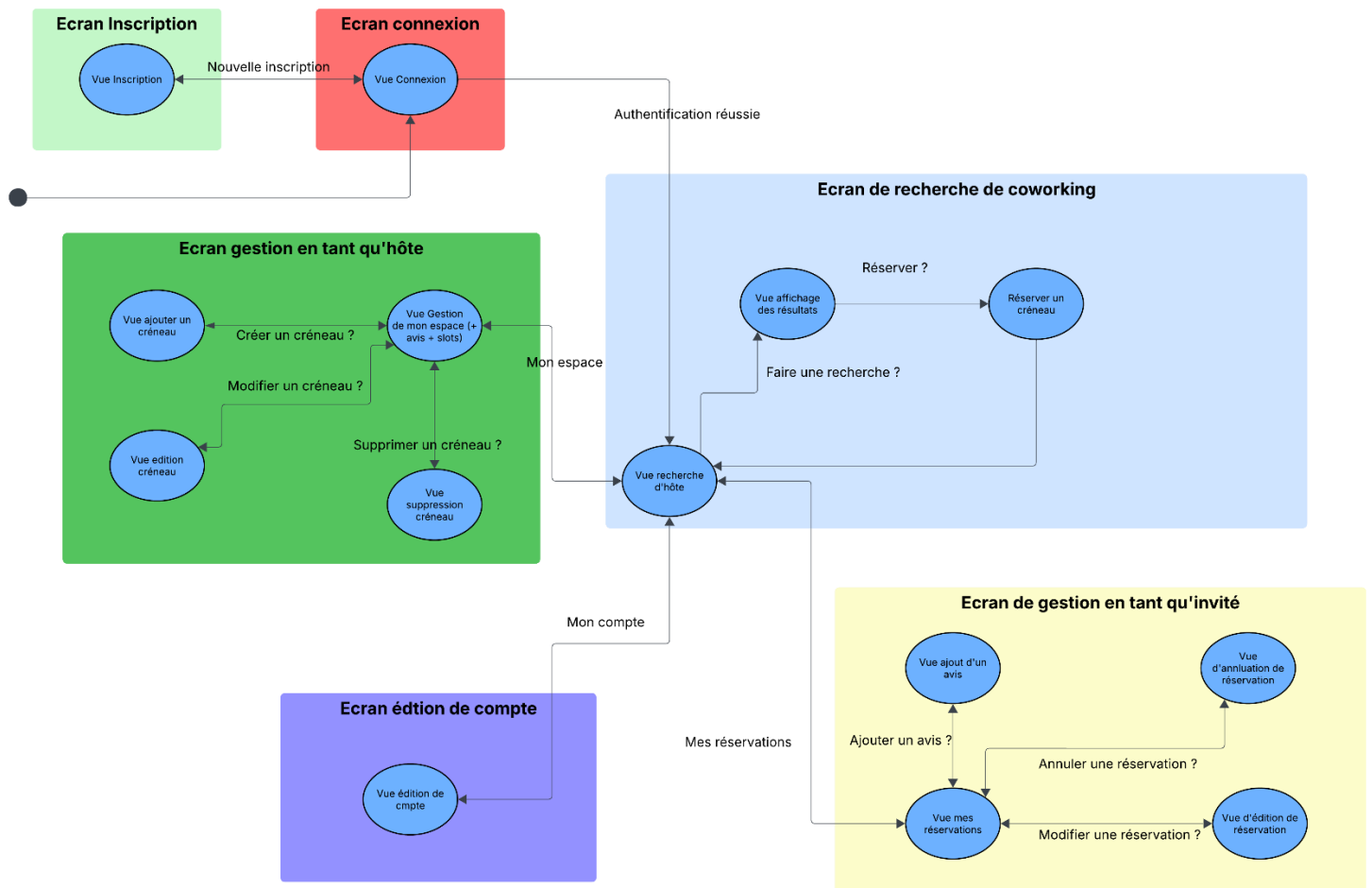


Figure 6 : Plan de navigation et du découpage par écran

Au cours de la réalisation de ce projet, un certain temps a été nécessaire pour réfléchir au meilleur moyen d'organiser les informations dans chacun des écrans. Il était important que tout soit atteignable en quelques clics. Pour ce faire, il a fallu dessiner des maquettes pour la plupart des écrans afin d'avoir une idée claire de l'objectif à atteindre avant de démarrer la réalisation. Ces maquettes sont disponibles dans les annexes.

11. Conclusion

Ce travail est parti d'un constat simple : le télétravail apporte des avantages mais peut engendrer un isolement social aux conséquences importantes sur la santé mentale des travailleurs.

Face aux limites des solutions existantes, nous avons conçu WorkTogetherly, une plateforme de proximité permettant, grâce à la géolocalisation, de mettre en relation des télétravailleurs et de partager un espace de travail. Ce travail nous a amenés à définir les besoins des utilisateurs au travers de personas, de user stories et de contraintes, à modéliser les données et à faire des choix technologiques pour construire une solution robuste en tenant compte des spécificités des différents supports.

WorkTogetherly apporte une réponse concrète à la problématique posée en permettant à des télétravailleurs de se retrouver dans un cadre de proximité. La plateforme recrée ce lien social informel qui manque tant au télétravailleur isolé, sans sacrifier les bénéfices du télétravail.

Plusieurs pistes d'améliorations sont apparues très clairement comme le développement d'une version SSR, un système de vérification d'identité et de paiement en ligne, l'intégration de notifications push natives, la possibilité de configurer si on veut recevoir ou non des emails de la plateforme ou une suppression du compte.

12. Bibliographie

Aseem. 2025. « The Result Pattern in C#: A Smarter Way to Handle Errors ». *Medium*. <https://medium.com/@aseem2372005/the-result-pattern-in-c-a-smarter-way-to-handle-errors-c6dee28a0ef0> (10 avril 2026).

bradygaster. « Présentation de SignalR ». <https://learn.microsoft.com/fr-fr/aspnet/signalr/overview/getting-started/introduction-to-signalr> (20 avril 2026).

claytonsiemens77. « Modèle CQRS - Azure Architecture Center ». <https://learn.microsoft.com/fr-fr/azure/architecture/patterns/cqrs> (20 avril 2026).

« Clean Coder Blog ». <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> (22 avril 2026).

« Client-Side Rendering (CSR) - Glossary | MDN ». 2025. *MDN Web Docs*. <https://developer.mozilla.org/en-US/docs/Glossary/CSR> (24 avril 2026).

« CQRS Validation with MediatR Pipeline and FluentValidation ». <https://www.milanjovanovic.tech/blog/cqrs-validation-with-mediatr-pipeline-and-fluentvalidation> (24 avril 2026).

« Dapper vs Entity Framework: When to Use Each in .NET ». 2025. *DEV Community*. <https://dev.to/alexnicolascode/dapper-vs-entity-framework-when-to-use-each-in-net-3e7j> (3 avril 2026).

« Différences entre SSR, CSR et SSG - laConsole ». <https://laconsole.dev/blog/differences-ssr-csr-ssg> (2 avril 2026).

European Agency for Safety and Health at Work. 2021. *Home-Based Teleworking and Preventive Occupational Safety and Health Measures in European Workplaces: Evidence from ESENER 3 : Report*. LU: Publications Office. doi:[10.2802/38822](https://doi.org/10.2802/38822).

« Générateur de site statique (SSG) - Glossaire | MDN ». 2026. *MDN Web Docs*. <https://developer.mozilla.org/fr/docs/Glossary/SSG> (3 mai 2026).

guardrex. « ASP.NET Core Blazor Render Modes ». <https://learn.microsoft.com/en-us/aspnet/core/blazor/components/render-modes?view=aspnetcore-10.0> (6 avril 2026).

He, Tun, Lijing Wei, Melody S. Goodman, José A. Pagán, Adolfo G. Cuevas, et Jemar R. Bather. 2026. « Remote work and loneliness: Evidence from a nationally representative sample of employed U.S. adults ». *Journal of Affective Disorders* 393: 120456. doi:[10.1016/j.jad.2025.120456](https://doi.org/10.1016/j.jad.2025.120456).

« Impact de la crise COVID sur le télétravail | Beswic ».

<https://beswic.be/fr/themes/teletravail/impact-de-la-crise-covid-sur-le-teletravail> (10 avril 2026).

« Isolement : « Ce qui nous relie aux autres au travail, c'est l'activité » | Anact ».

<https://www.anact.fr/isolement-ce-qui-nous-relie-aux-autres-au-travail-cest-lactivite> (10 avril 2026).

Jakobowicz, Emmanuel. 2026. « Comprendre les SGBD relationnels : comparatif complet et usages - Stat4decision ». <https://www.stat4decision.com/fr/sghd-relationnels/> (20 avril 2026).

Jones, Michael B., John Bradley, et Nat Sakimura. 2015. *JSON Web Token (JWT)*. Internet Engineering Task Force. Request for Comments. doi:[10.17487/RFC7519](https://doi.org/10.17487/RFC7519).

« Le télétravail et les risques de TMS | Beswic ». <https://beswic.be/fr/blog/le-teletravail-et-les-risques-de-tms> (21 avril 2026).

« Présentation des JSON Web Token (JWT) ». 2020. *ionos Digital Guide*.

<https://www.ionos.fr/digitalguide/sites-internet/developpement-web/json-web-token-jwt/> (23 avril 2026).

Roumane, Abderrahmane. 2023. « Les Fondements de La Clean Architecture: Une Voie Vers Un Code Durable ». *Medium*.

<https://medium.com/@abderrahmane.roumane.ext/les-fondements-de-la-clean-architecture-une-voie-vers-un-code-durable-1542f91f9d0a> (19 avril 2026).

SamMonoRT. « Fournisseurs de base de données - EF Core ».

<https://learn.microsoft.com/fr-fr/ef/core/providers/> (20 avril 2026).

Sauro, William, et Professeur Karl Colin. « Quel est l'impact du télétravail sur la productivité ? »

« Secure Storage in Android: SharedPreferences, DataStore, and Keystore Explained | by Charles Raj Iruthayaraj | Medium ». <https://medium.com/@charles-raj/secure-storage-in-android-sharedpreferences-datastore-and-keystore-explained-bf59474aa382> (1 mai 2026).

« Server-Side Rendering (SSR) - Glossary | MDN ». 2025. *MDN Web Docs*.

<https://developer.mozilla.org/en-US/docs/Glossary/SSR> (6 mai 2026).

« Télétravail | SPF Emploi - Travail et Concertation sociale ».

<https://emploi.belgique.be/fr/themes/bien-etre-au-travail/teletravail> (5 mai 2026).

« WebAssembly ». <https://webassembly.org/> (4 mai 2026).

« WebAssembly | MDN ». 2025. *MDN Web Docs*.

<https://developer.mozilla.org/fr/docs/WebAssembly> (4 mai 2026).

13. Annexes

Annexe 1 : Maquettes

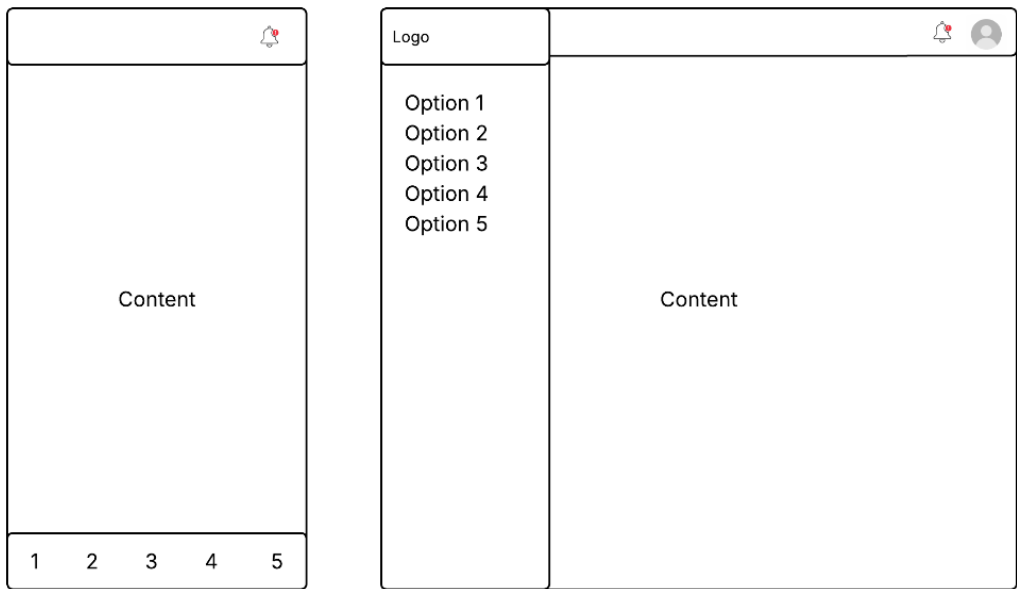


Figure 7 : Maquette de navigation

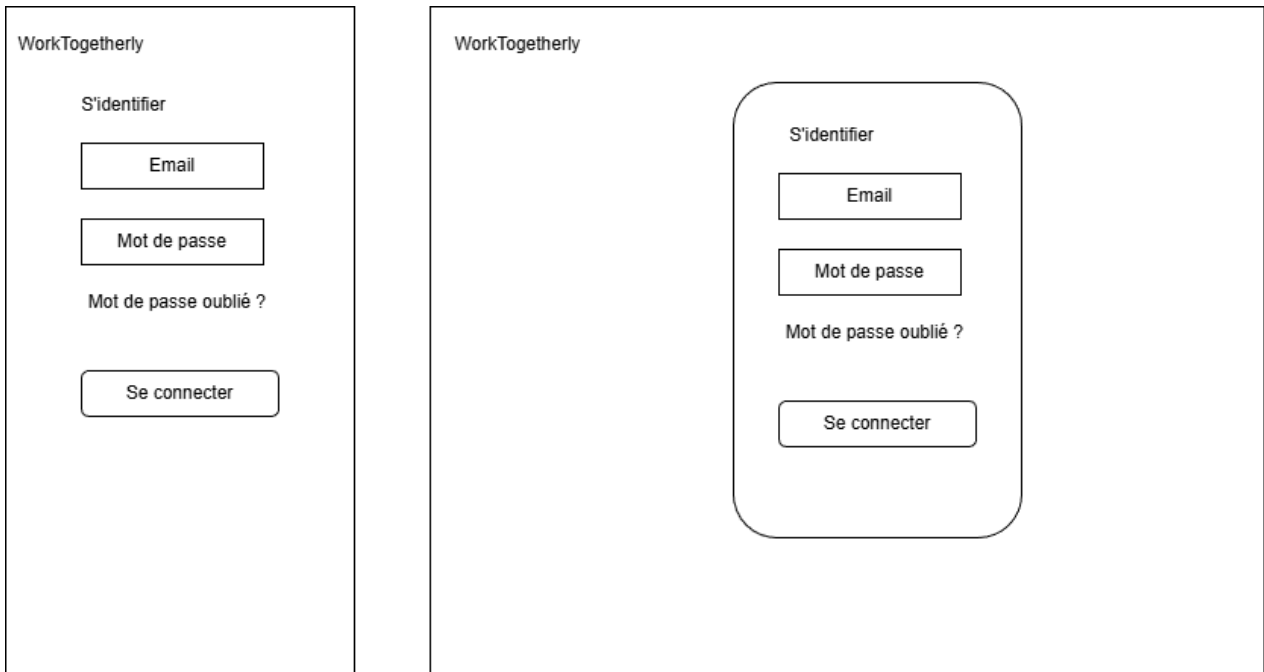


Figure 8: Maquette de l'écran de connexion

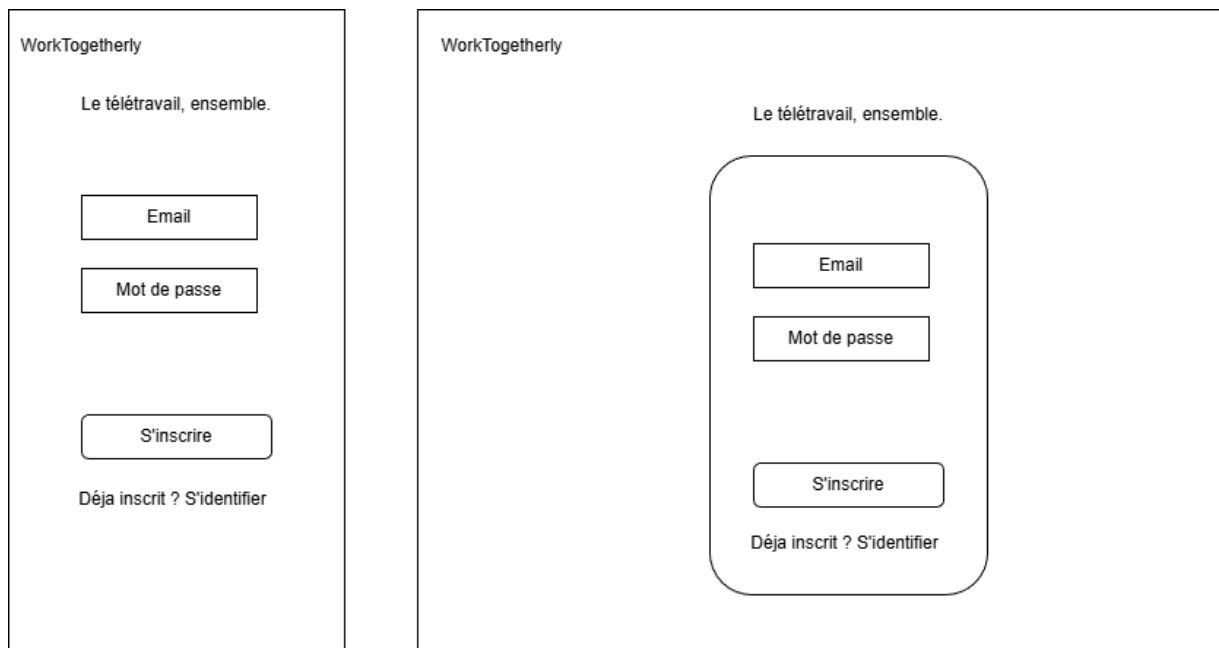


Figure 9 : Maquette de l'écran de création de compte

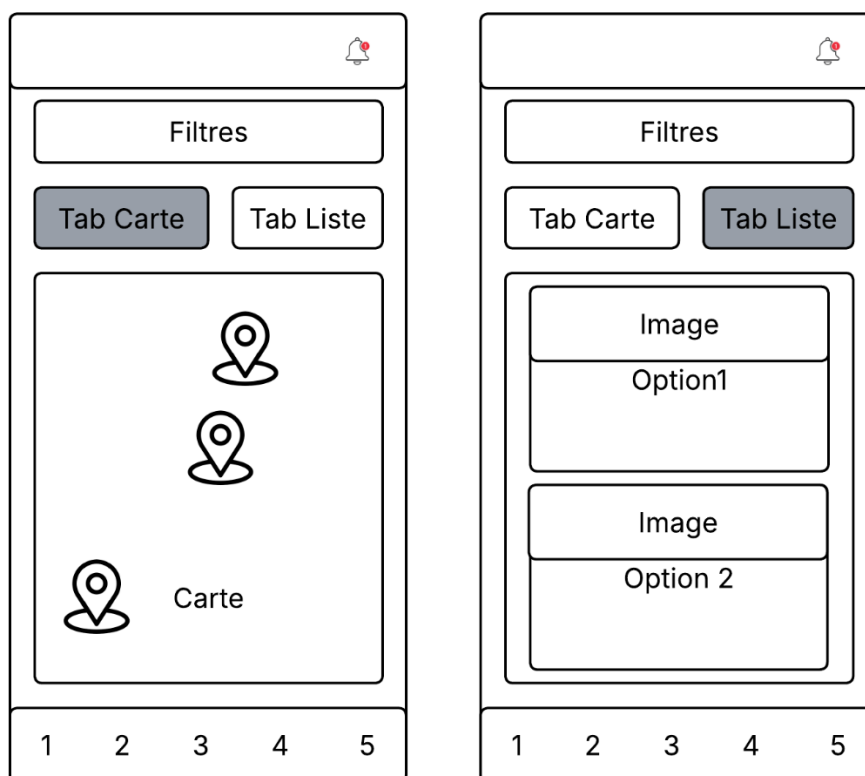


Figure 10 : Maquette de l'écran de recherche d'un espace de travail sur mobile

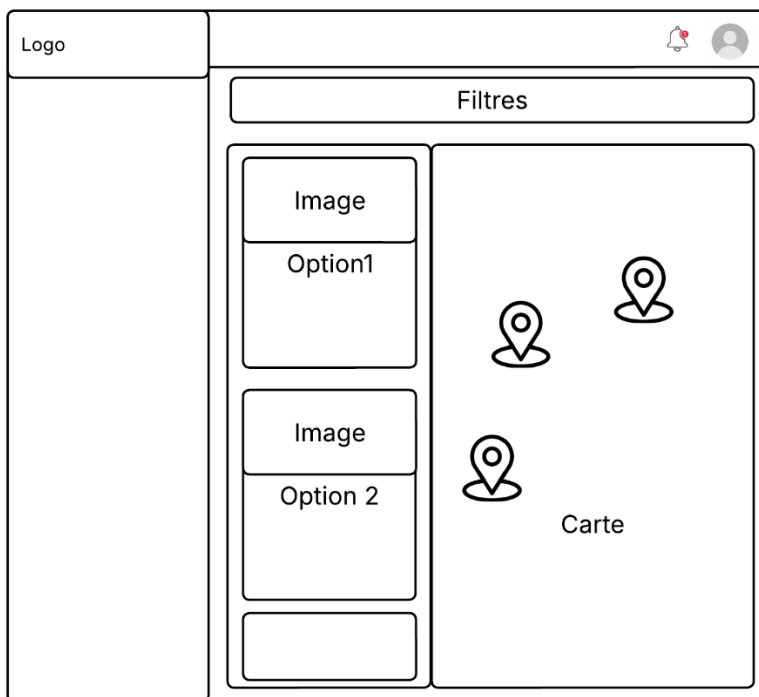


Figure 11: Maquette de l'écran de recherche d'un espace de travail sur ordinateur

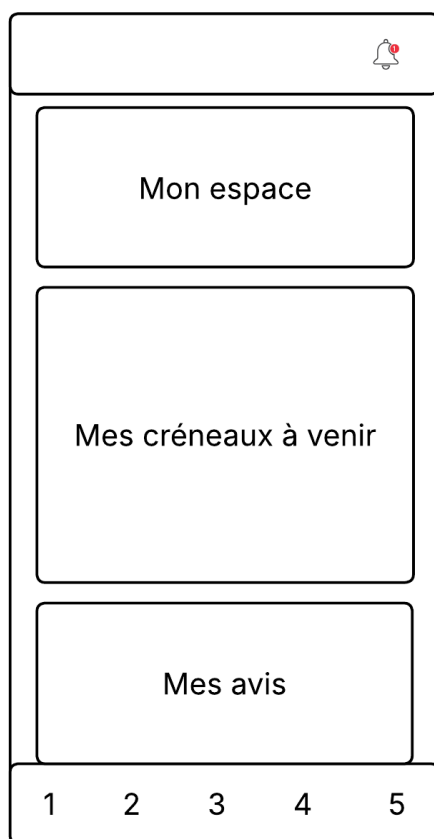


Figure 12 : Maquette de l'écran de gestion de l'espace de travail sur mobile

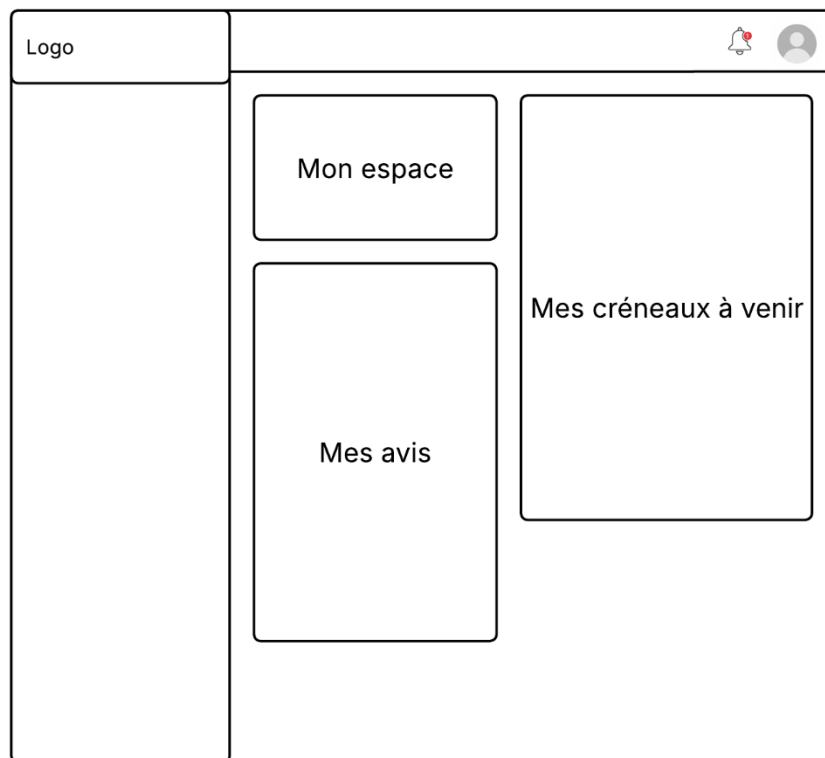


Figure 13 : Maquette de l'écran de gestion de l'espace de travail sur ordinateur

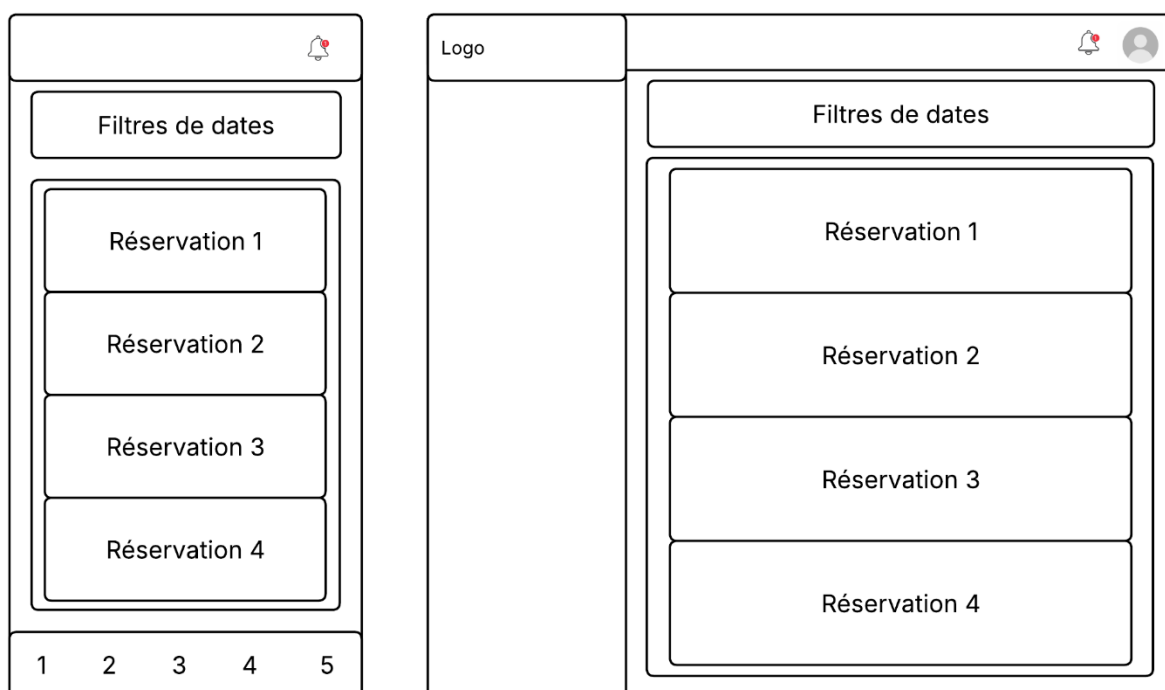


Figure 14 : Maquette d'écran de gestion des réservations

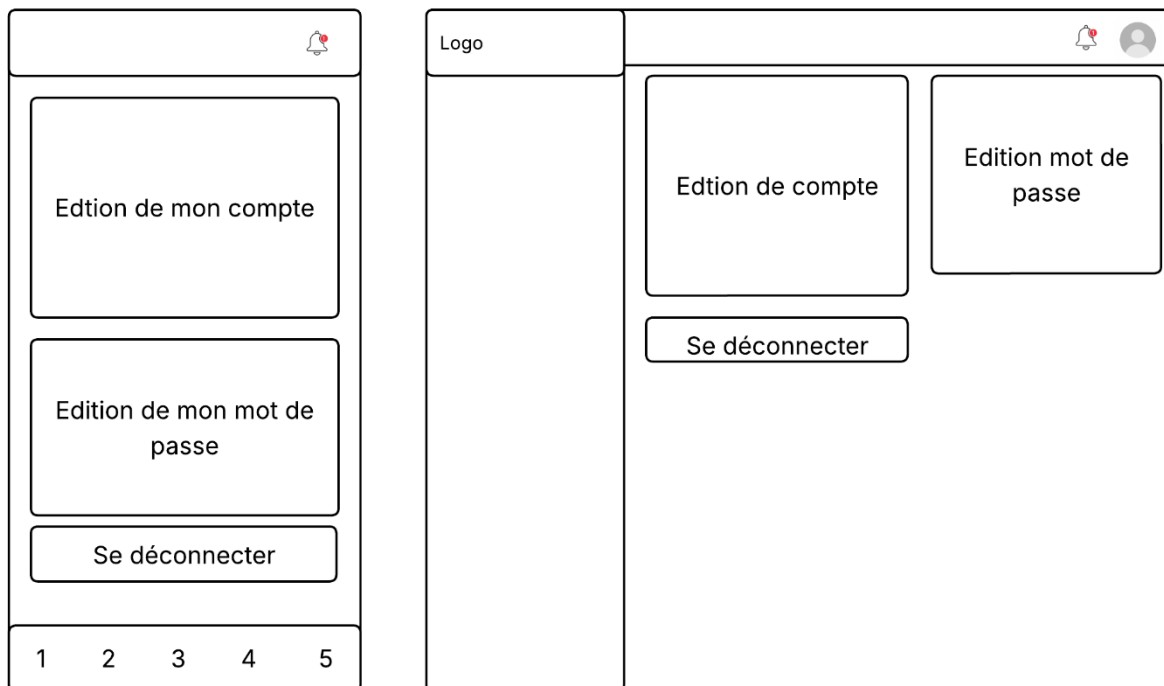


Figure 15 : Maquette de l'écran de gestion de compte

Annexe 2 : Diagramme d'authentification

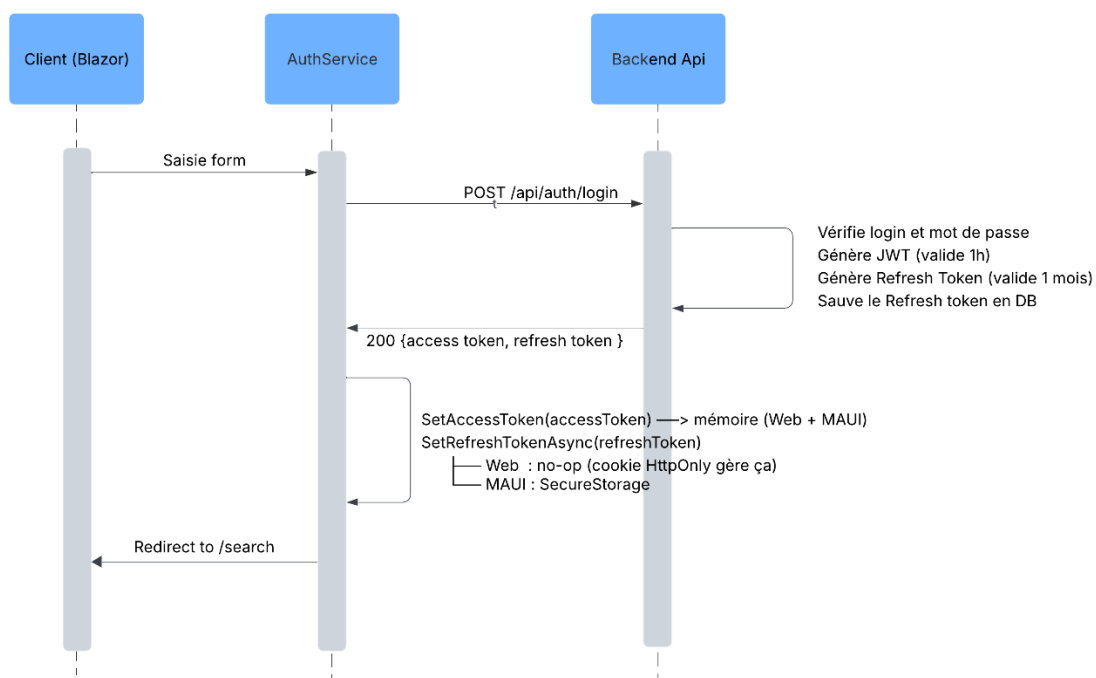


Figure 16: Diagramme de séquence du processus d'authentification

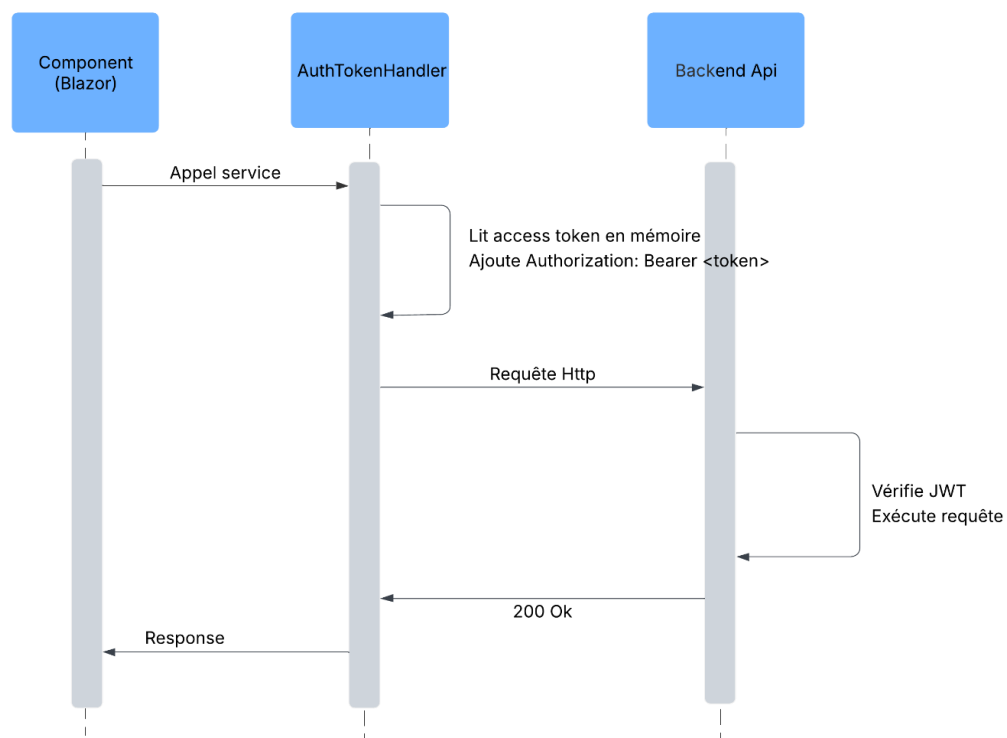


Figure 17: Diagramme de séquence d'une requête d'un utilisateur connecté

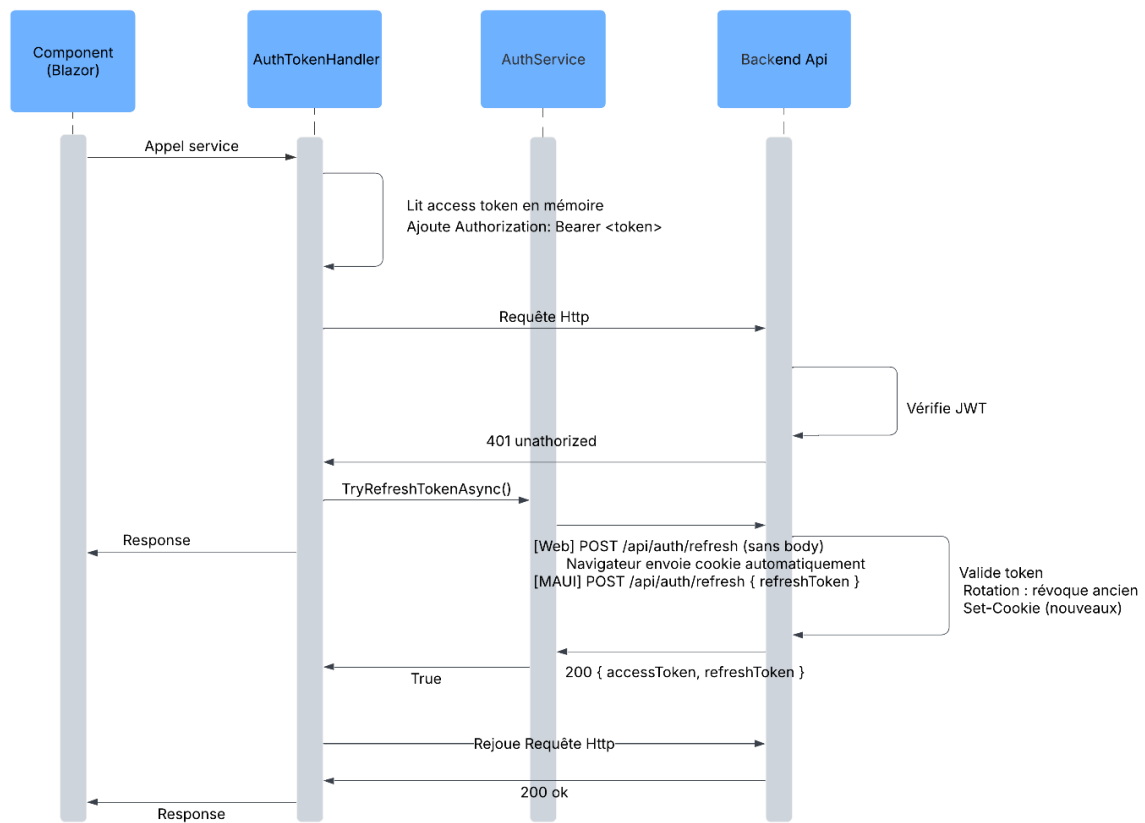


Figure 18: Diagramme de séquence du rafraichissement d'un jeton d'authentification

Annexe 3 : Analyse MoSCoW

Must Have

- Le système doit permettre à une personne de se créer un compte avec son adresse électronique et un mot de passe.
- Le système doit permettre à l'utilisateur d'encoder ses informations de base nécessaires au bon fonctionnement de l'application.
- Le système doit permettre à un utilisateur de s'authentifier en entrant son adresse électronique et son mot de passe.
- Le système doit permettre à un utilisateur de définir un espace de travail.
- Les espaces de travail doivent avoir une description, un titre et une capacité.
- Le système doit permettre d'effectuer une recherche autour de la position d'un utilisateur entrée manuellement ou automatiquement évaluée (via géolocalisation).
- Le système doit afficher les résultats dans une liste ou sur une carte.
- Le système doit afficher une position approximative des hôtes tant que la réservation n'est pas faite pour respecter la vie privée des utilisateurs.
- Le système doit permettre de sélectionner un résultat pour afficher les détails d'un espace de travail.
- Le système doit permettre à l'hôte d'indiquer son heure d'arrivée.
- Le système doit permettre aux hôtes de laisser une note de 1 à 5 sur leurs réservations.
- Le système doit permettre aux hôtes de laisser un commentaire sur leurs réservations.
- Le système doit informer les hôtes de toutes les modifications faites dans les réservations de leurs slots.
- Le système doit informer les invités de toutes les modifications faites dans les créneaux qu'ils ont réservés.

13.1.1.1. *Should Have*

- Le système devrait proposer un mécanisme de récupération de mot de passe via envoi d'un mail sur l'adresse utilisée dans le compte.
- Le système devrait permettre à l'utilisateur de changer son mot de passe au sein de l'application.
- Le système devrait permettre à l'utilisateur de changer ses informations de base (nom, prénom, ...).
- Le système devrait permettre de définir les caractéristiques de son lieu de travail (non-fumeur, animaux de compagnie acceptés).
- Le système devrait permettre à l'hôte de mettre une photo de l'espace de travail.
- Le système devrait permettre à un hôte d'indiquer le matériel qu'il possède.

- Le système devrait permettre de filtrer les résultats suivant différents critères.
- Le système devrait informer l'invité des changements significatifs sur sa réservation.
- Un utilisateur doit pouvoir consulter les évaluations qui lui ont été laissées.
- Le système devrait envoyer des communications sous la forme de messages internes à l'app.
- Le système pourrait envoyer les communications sous la forme de courriers électroniques.

13.1.1.2. *Could Have*

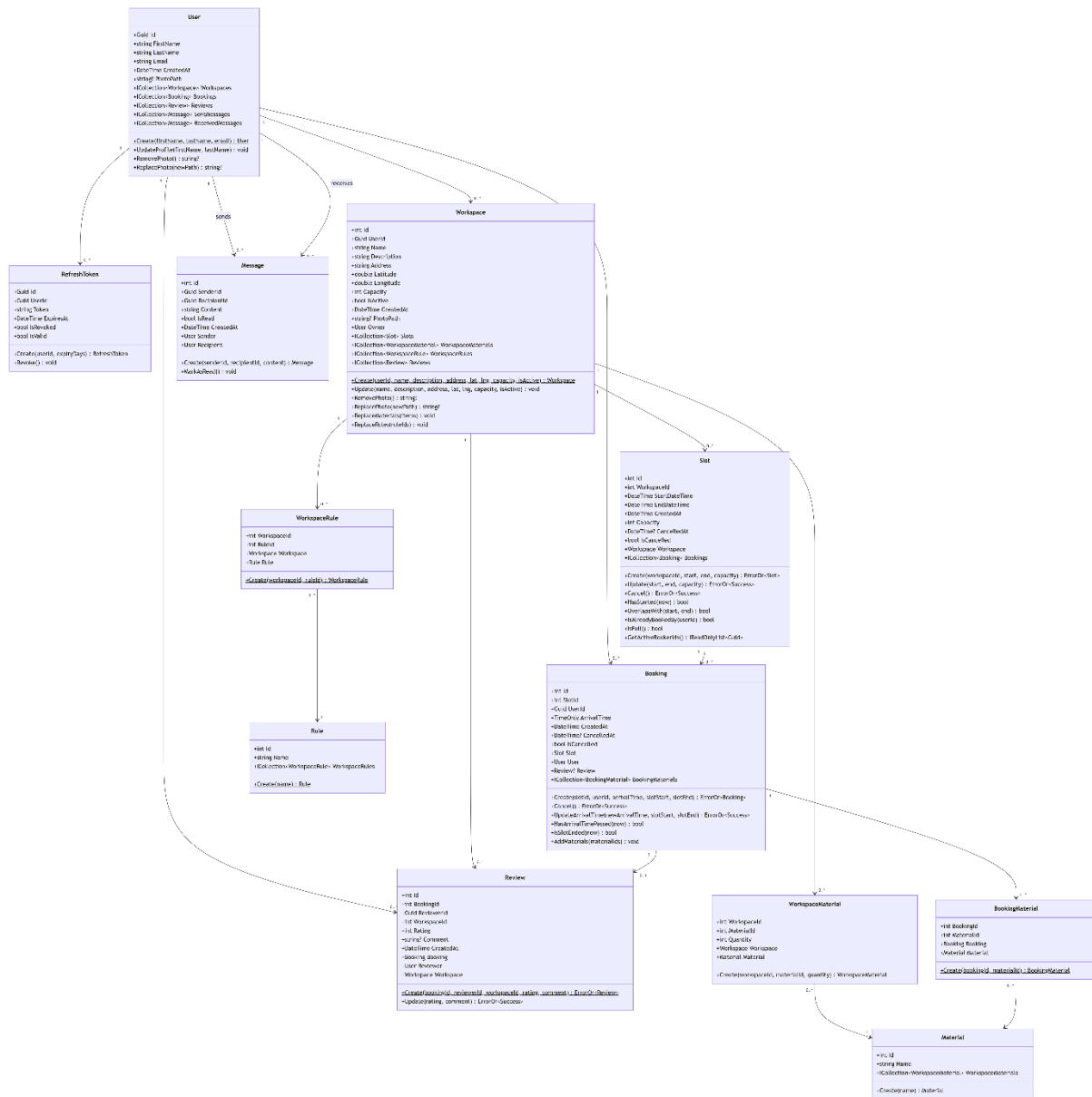
- Le système pourrait proposer la suppression d'un compte utilisateur.
- Le système pourrait proposer une identification à deux facteurs.
- Le système pourrait proposer la désactivation de certains utilisateurs.
- Le système pourrait permettre à l'utilisateur de configurer plusieurs espaces de travail pour facilement proposer différents lieux les différents jours.
- Le système pourrait permettre un mode non-connecté permettant de visualiser les hôtes aux alentours.
- Le système pourrait permettre de mettre certains hôtes en favori.
- Le système pourrait proposer à l'utilisateur d'ajouter les réservations à son calendrier (Google, Outlook, ...).
- Le système pourrait proposer un système de paiement en ligne avec commission pour la plateforme.
- Le système pourrait proposer de répondre à un commentaire.
- Le système pourrait envoyer les communications sous la forme de notifications (dans le navigateur ou dans l'application mobile native).

13.1.1.3. *Won't Have*

- Le système n'aura pas de création de compte via une plateforme sociale (Facebook, Google, ...).
- Le système n'aura pas de système pour vérifier l'identité de chacune des personnes se créant un compte via documents d'identité, application d'identité mobile, carte d'identité, ...
- Le système n'ira pas chercher automatiquement dans le calendrier de l'hôte l'existence de rendez-vous à l'extérieur pour déterminer ses disponibilités.
- Le système n'aura pas de système de filtrage automatique des commentaires (modération automatique).
- Le système ne proposera pas de rôle de modérateur pour vérifier le contenu des commentaires.
- Le système ne proposera pas de système permettant de signaler un commentaire.

- Le système n'aura pas de système permettant de supprimer ou cacher un commentaire.
- Le système n'aura pas de messagerie interne entre les utilisateurs.

Annexe 4 : Diagramme de classe



Annexe 5 : Enquête sur le télétravail et la santé mentale

09/05/2026, 14:51

L'impact du télétravail sur la santé mentale

L'impact du télétravail sur la santé mentale

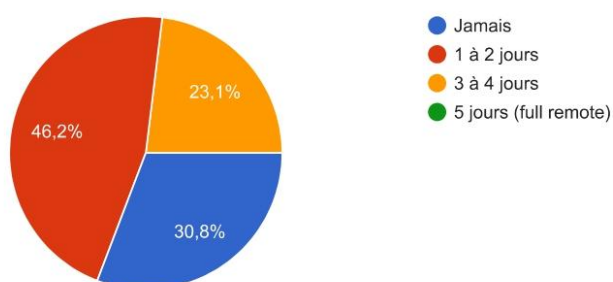
13 réponses

[Publier les données analytiques](#)

Tu télétravailles combien de jours par semaine en moyenne ?

[Copier](#)

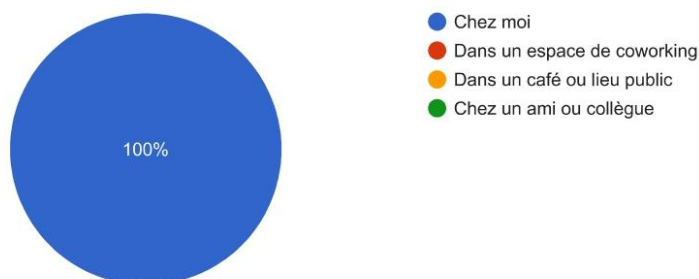
13 réponses



Où télétravailles-tu le plus souvent ?

[Copier](#)

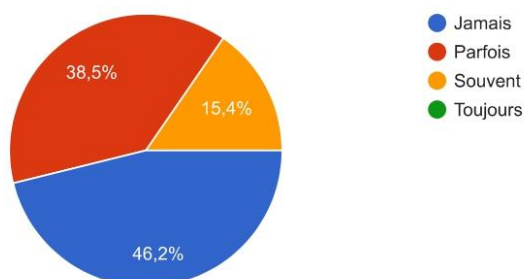
13 réponses



Est-ce que tu ressens de l'isolement social quand tu télétravailles ?

[Copier](#)

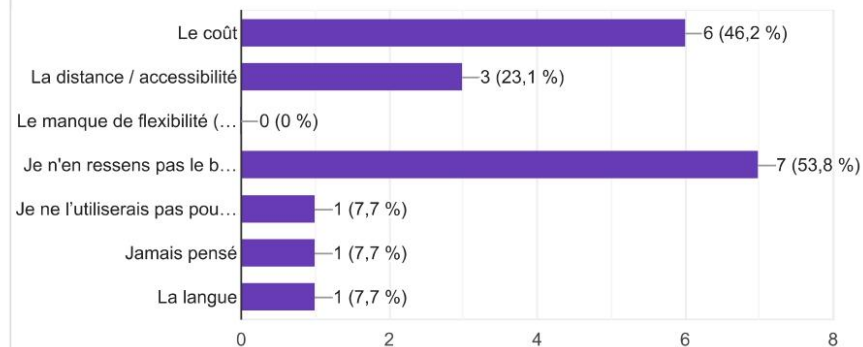
13 réponses



Qu'est-ce qui t'empêche d'utiliser un espace de coworking ?

 Copier

13 réponses



Ce contenu n'est ni rédigé, ni cautionné par Google. - [Contacter le propriétaire du formulaire](#) - [Conditions d'utilisation](#) - [Règles de confidentialité](#)

Ce formulaire vous semble suspect ? [Signaler](#)

Google Forms

